

动态规划

我想这毫无疑问将是期中之前这门课最令人感到噩梦的一章, 因为动态规划并不是一个很 trivial 的方法, 不像回溯、分治以及之后的贪心自然直观。但我相信, 经过本讲的讲解与推导, 以及诸位自己的仔细思考, 至少对这门课的考试、未来的研究和工作都能打下坚实的基础。

8.1 动态规划的背景

8.1.1 动态规划的来由

1966 年, Dynamic Programming 登上国际顶级期刊《Science》, 理查德·贝尔曼 (Richard Bellman) 为唯一作者。贝尔曼在他的自传中写道: “1950 年秋季, 我在兰德公司工作。我的第一个任务是为多阶段决策过程找到一个名称。”

“一个有趣的问题是, “动态规划” 这个名字是从哪里来的? 20 世纪 50 年代不是搞数学研究的好年代。当时在华盛顿有一位非常有趣的人, 他叫威尔逊 (Charles E. Wilson, 1953-1957 年任美国国防部长), 是当时的美国国防部长, 他对 “研究” 这个词有病态的恐惧和憎恨。提到研究, 他的脸会涨得通红, 如果有人在他面前用 “研究” 这个词, 他就会变得很暴力。你可以想象他对数学这个词的感受。兰德公司受雇于空军, 而空军的老板基本上是威尔逊。因此, 我觉得我必须做点什么来瞒过威尔逊和空军, 不让他们知道我实际上是在兰德公司做数学研究。”

“所以, 给这个研究起个什么名字好呢? 首先, 我对 planning, decision making 和 thinking 比较感兴趣。但是由于种种原因, planning 这个词并不合适。因此, 我决定使用 programming 这个词。我想让大家明白, 这是动态的, 是多级的, 是时变的——我想, 这个名字可以起到一石二鸟的效果。作为形容词, dynamic 还有一个很有趣的特性, 那就是 dynamic 这个词不可能有贬义。我们不可能想出一些可能使它具有贬义的组合。因此, 我认为 dynamic programming 是个好名字。这是一件连国会议员都不会反对的事情。所以我把它作为我研究活动的 “保护伞。”¹

8.1.2 基础问题: 爬楼梯

为了理解动态规划的思想, 我们先来看一个简单的问题: 假如你要爬楼梯, 需要爬 n 阶才能到达楼顶, 每次可以爬 1 或 2 个台阶, 你需要计算有多少种不同的方法。

这个问题当然可以用排列组合来做。假设有 k 步一次跨了两个台阶, 那么剩下的 $n - 2k$ 个台阶就要一步一步地爬, 总共需要走 $n - k$ 步。只要从中选出哪 k 步跨两级, 就确定了一种走法。因此, 记爬 n 阶楼梯的方法数为 $f(n)$, 便有 $f(n) = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n-k}{k}$ 。不过, 我们还是希望用算法来解决这个

¹命名故事见 Bellman 自传 *Eye of the Hurricane* 的摘录: Stuart Dreyfus, “Richard Bellman on the Birth of Dynamic Programming”, *Operations Research*, 50(1):48-51, 2002。以上内容摘自[这个知乎文章](#)。

问题。最直接的方案自然是回溯法，如果本题要求列出所有方案，回溯是合理的，但我们只要一个总数，逐个列举就可能做了很多不必要的工作。

先尝试分治

上一讲介绍过分治法，我们也可以试着把楼梯分成两半。先考虑 $n = 2m$ ，并按照是否踩到中间的第 m 级台阶，将走法分成两类。踩到了这一阶，就相当于先爬 m 阶，再爬 m 阶，前后任意两种走法都可以组合，共有 $f(m)^2$ 种。没有踩到这一阶，就一定是从第 $m - 1$ 阶跨到了第 $m + 1$ 阶；除去中间这一步，两边都还要爬 $m - 1$ 阶，共有 $f(m - 1)^2$ 种。因此

$$f(2m) = f(m)^2 + f(m - 1)^2 \quad (m \geq 1).$$

当 $n = 2m + 1$ 时，也可以按是否踩到第 m 阶来分类：踩到了，两边分别要爬 m 阶和 $m + 1$ 阶；跨过去了，两边则分别要爬 $m - 1$ 阶和 m 阶。于是

$$f(2m + 1) = f(m)f(m + 1) + f(m - 1)f(m) \quad (m \geq 1).$$

按这两个公式递归计算，就能得到答案。如果直接使用上述递推式进行计算，考虑 $n = 2m + 1$ 的情况，复杂度递推式为 $T(n) = 4T(n/2 + O(1)) + O(1)$ ，复杂度是 $O(n^2)$ 的，并不令人满意；当 $n = 2m$ 时，虽然时间复杂度的递推式是 $T(n) = 2T(n/2 + O(1)) + O(1)$ ，看起来很美好，但可能 $n/2$ 是奇数，因此复杂度可能仍然很高。

从递推式到动态规划

分治法的失败迫使我们思考新的解法，或许可以对这个问题的结构进行一些观察。为了算法的效率性，利用子问题的组合来帮助解决最后的问题是一条合适的道路。虽然对半分不适合，但这个问题的解应该可以从小问题的解出发得到最后整个问题的解。解的最后一步或许是一个好的出发点：要到达第 n 阶，要么从第 $n - 1$ 阶跨一级上来，要么从第 $n - 2$ 阶跨两级上来。前一种情况有 $f(n - 1)$ 种走法，后一种有 $f(n - 2)$ 种，加起来就是全部走法：

$$f(n) = f(n - 1) + f(n - 2) \quad (n \geq 2).$$

这个递推关系是不是很熟悉？没错，这就是斐波那契数列的定义！我们可以很自然地把这个表达式转为递归计算的代码实现，其中 $f(0) = f(1) = 1$ 是初值。然而这种实现会产生一个严重的问题：在计算 $f(n - 1)$ 的过程中，我们已经算出了 $f(n - 2)$ ，返回以后却又要为第二项重新计算一次。再往下展开，类似的重复计算还会大量发生。这些重复计算到底有多少？设求出 $f(n)$ 总共需要调用函数 $C(n)$ 次，则根据递推式直接可以得到 $C(0) = C(1) = 1$ ， $C(n) = C(n - 1) + C(n - 2) + 1$ ，显然 $C(n)$ 关于 n 是指数级别的！这是完全不可接受的，这会使得算法的时间复杂度是指数级别的！

既然浪费来自同一个值反复计算，解决办法也很自然：每算出一个 $f(k)$ ，就把它保存在数组里，以后再用到时直接读取即可。这就是**记忆化递归**。我们甚至可以干脆从已知的 $f(0)$ 、 $f(1)$ 开始，顺着递推式算出 $f(2)$ 、 $f(3)$ 、 $\dots$ 、 $f(n)$ ，这样连递归调用也省去了，称为**自底向上填表**。如果只关心最后的答案，由于每次只需要前两项，甚至不必保留整个数组，用两个变量就够了。下面给出了基于上述思想的伪代码，其中 *Last* 和 *NextToLast* 分别保存 $f(i - 1)$ 和 $f(i - 2)$ ，不难分析得到时间复杂度为 $O(n)$ 时间。

算法 8.1 爬楼梯：迭代计算**输入：** 整数 $n \geq 0$ **输出：** 爬 n 阶楼梯的方法数 $f(n)$

```

1: 若  $n \leq 1$ , return 1
2:  $Last \leftarrow 1$ ;  $NextToLast \leftarrow 1$ 
3: for  $i \leftarrow 2$  to  $n$  do
4:    $Answer \leftarrow Last + NextToLast$ 
5:    $NextToLast \leftarrow Last$ ;  $Last \leftarrow Answer$ 
6: end for
7: return  $Answer$ 

```

8.1.3 从计数到最优化

总结一下上面的讨论：在爬楼梯这一简单而经典的问题中，通过对最后一步两种情况的分类，将整个问题的解转化为了两个子问题解的求和，即有一个从子问题的解到原问题的解的递推公式，然后通过记忆化的方式求解递推式。当然在爬楼梯问题中，我们的任务只是要求数出有多少种方法。在接下来要讨论的问题中，我们希望在许多可行方案中找出最好的一个，例如让收益最大，或让花费最小。这类问题仍然可以尝试用子问题的答案构成原问题的答案：一个问题，它的最优解可以表达为一些合适的子问题的最优解的递推关系，则称这一问题具有**最优子结构性质**，因为大问题的最优解可以直接依赖于小问题的最优解。然后求解这一递推式，通过记忆化的方法，使用迭代算法而非费时的递归算法避免冗余计算，得到一个时间复杂度令人满意的算法，这就是动态规划的基本想法。

通常按如下 4 个步骤设计动态规划算法：

1. 分析最优解由哪些部分构成，确定需要求解的子问题；
2. 写出子问题最优值之间的递推关系；
3. 给出初值，并选择计算顺序，保证能自底向上计算出原问题的最优值；
4. 利用计算时保存的信息，恢复一个最优解。

步骤 1-3 是动态规划算法求解问题的基础。如果仅仅需要一个最优解的值，而非解本身，可以忽略步骤 4（上面的例子就没有这一步骤，当然上面的例子也不是最优化问题，但接下来马上就可以看到包含步骤 4 的例子）。如果确实要做步骤 4，有时就需要在执行步骤 3 的过程中维护一些额外信息，以便用来构造一个最优解。

接下来将开始正式的旅途，我们将会从易到难，观察一些经典的例子的解法。动态规划是一个经验和技巧都很重要的算法设计思想，因此见识很多不同风格、不同特点的例子并加以自己的思考是非常重要的。

8.2 加权独立集合问题**8.2.1 问题描述**

第一个问题称之为加权独立集合问题。可以从一个带故事背景的问题出发：假如你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的

房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。给定一个代表每个房屋存放金额的非负整数数组，计算你不触动警报装置的情况下，一夜之内能够偷窃到的最高金额。

如果把每间房屋看成一个顶点，相邻两间房屋之间连一条边，就可以把这题画成一条路径。将顶点依次记为 $v_1, \dots, v_n$ ，第 i 间房屋里的现金就是顶点 v_i 的权重 w_i ，相邻顶点之间有边 (v_i, v_{i+1}) 。

在图中，一个顶点集合如果没有任何两个顶点相邻，就称为**独立集合**。题目要求相邻房屋不能同时选择，也就是要求选出的顶点构成独立集合；要偷到尽可能多的现金，就要使这些顶点的权重之和最大。这样得到的问题称为**最大加权独立集合问题**，本题先研究图是一条路径的情况。

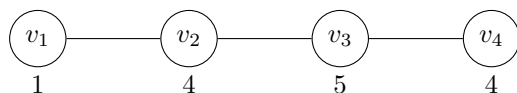


图 8.1 权重为 1, 4, 5, 4 的路径图。

图中共有 8 个独立集合：空集、四个单点集，以及 $\{v_1, v_3\}$ 、 $\{v_1, v_4\}$ 、 $\{v_2, v_4\}$ 三个两点集。权重最大的是 $\{v_2, v_4\}$ ，总权重为 8，对应选择第 2、4 家。

8.2.2 最优子结构

最直接的做法是枚举每个顶点选不选，但这样最多要检查 2^n 个子集。也可以尝试从中间分成两段，分别求最优解，不过两段的方案可能同时选中了分界处相邻的两个顶点，拼在一起便不合法。为了避开这一冲突，可以做一个简单的自顶向下的思维实验：如果你拿到一个路径图，那么对于这一问题的解首先会想什么？你应该想知道最后一个点是不是在最优解中，因为只有知道了这个点不在才能进一步向前继续构造出整个问题的解！所以我们先确定最后一个顶点 v_n 选不选，再看剩下的部分。记前 i 个顶点构成的路径图为 G_i ，设 S 是原图的一个最优解。

1. 若 $v_n \notin S$ ，那么 S 就是 G_{n-1} 的一个独立集合，而且必须是 G_{n-1} 的最大加权独立集合。否则，可以用 G_{n-1} 中权重更大的独立集合替换 S ，得到一个更好的解，这与 S 是最优解矛盾。
2. 若 $v_n \in S$ ，它的邻点 v_{n-1} 就不能入选。去掉 v_n 后，剩下的顶点都在 G_{n-2} 中，而且必须构成 G_{n-2} 的最大加权独立集合。否则，将这部分换成更好的解，再加上 v_n ，也会改进原方案。

由此，整个问题的最优解依赖于前 $n-1$ 和 $n-2$ 个点构成的子图的最优解，或者说，这两个子问题的最优是构成原问题最优的基础，这就是这一问题对应的最优子结构性质。设 G_i 的最优加权独立集合的权重之和为 W_i ，可以写出递推关系：

$$W_n = \max\{W_{n-1}, W_{n-2} + w_n\},$$

这里是最后一步的递推关系，事实上可以不断利用这一递推关系自顶向下向前构建出整个问题的解，即有更一般的表达

$$W_i = \max\{W_{i-1}, W_{i-2} + w_i\},$$

其中 $i = 2, 3, \dots, n$ ， $W_0 = 0$ 。于是，到目前为止完成了动态规划四步框架的前两步，接下来需要写代码实现这一算法。

算法 8.2 路径图的最大加权独立集合：填表

输入： 顶点依次为 $v_1, \dots, v_n$, 权重 $w_1, \dots, w_n \geq 0$

```

1:  $W[0] \leftarrow 0$ 
2: if  $n \geq 1$  then
3:    $W[1] \leftarrow w_1$ 
4: end if
5: for  $i \leftarrow 2$  to  $n$  do
6:    $W[i] \leftarrow \max\{W[i-1], W[i-2] + w_i\}$ 
7: end for
8: return  $W[n]$  和数组  $W$ 

```

根据上述算法, 不难计算出图 8.1 的例子返回的数组为 $W = (0, 1, 4, 6, 8)$ 。例如前三个点的最优值是 $\max\{4, 1 + 5\} = 6$, 而加入第四个点时又比较 6 与 $4 + 4$, 最终选择后者。不难分析上述算法的时间复杂度为 $\Theta(n)$, 因为只需要从 1 到 n 遍历一次, 且遍历过程只做常数时间的比较和加法。

8.2.3 解的重构

数组 W 给出了最优权重, 但还没有列出选了哪些顶点。可以从 $i = n$ 开始, 反过来检查递推式恢复出一个最优解。

如果 $W_i = W_{i-1}$, 说明即使不选 v_i , 前 $i-1$ 个顶点也能给出当前的最优值, 因此可以跳过 v_i , 继续在前 $i-1$ 个顶点中寻找。否则, 最优值一定来自 $W_{i-2} + w_i$, 我们就选中 v_i , 并跳过它的邻点 v_{i-1} , 接着在前 $i-2$ 个顶点中寻找。这样从后往前, 便能把选择的顶点逐个找出来。如果两种选择的权重相等, 下面的代码采用不选 v_i 的方案 (事实上也可以选择 v_i , 都可以得到最优解, 因此最优解不唯一)。

算法 8.3 路径图的最大加权独立集合：恢复一个方案

输入： 已计算得到的 W

```

1:  $S \leftarrow \emptyset$ ;  $i \leftarrow n$ 
2: while  $i \geq 1$  do
3:   if  $W[i] = W[i-1]$  then
4:      $i \leftarrow i-1$ 
5:   else
6:     将  $v_i$  加入  $S$ ;  $i \leftarrow \max\{0, i-2\}$ 
7:   end if
8: end while
9: return  $S$ 

```

例如, 在前面的四个顶点中, $W[4] = 8 > W[3] = 6$, 因此选中 v_4 , 跳过 v_3 。接着, $W[2] = 4 > W[1] = 1$, 因此选中 v_2 , 最终得到 $\{v_2, v_4\}$ 。

进一步地, 这一问题中每个顶点都有非负的权重, 假设允许顶点的权重为负数, 那么上面的解法是否仍然适用? 很显然, 递推关系仍然成立, 但初值需要改动。因为如果只有一个顶点, 且它的权重

为负数，那么选它会得到负数，而不选则得到 0，因此应当取 $W[1] = \max\{0, w_1\}$ 。

下面我们使用类似的思想分析经典的最大子序列和问题：

例题 8.1 (最大子序列和问题). 给定非空数组，求和最大的非空连续子序列。请用动态规划写出递推关系，并给出求最大和及对应子序列的算法。

解答. 考虑一个以 a_i 结尾的非空连续子序列。它或者只有 a_i 这一项，或者是在一个以 a_{i-1} 结尾的连续子序列后面加上 a_i 。既然末尾的 a_i 已经确定，后一种情况下当然应当让前面的和尽可能大。因此，设 $E[i]$ 表示以 a_i 结尾的最大连续子序列和，就有

$$E[1] = a_1, \quad E[i] = \max\{a_i, E[i-1] + a_i\}.$$

最优的子序列可能在任何位置结束，所以还要在所有 $E[i]$ 中取最大值。

算法 8.4 最大子序列和

输入： 数组 $a[1..n]$ ； $n = 0$ 时返回“无非空子序列”

```

1:  $e \leftarrow a_1$ ;  $start \leftarrow 1$ ;  $best \leftarrow a_1$ ;  $(L, R) \leftarrow (1, 1)$ 
2: for  $i \leftarrow 2$  to  $n$  do
3:   if  $e < 0$  then
4:      $e \leftarrow a_i$ ;  $start \leftarrow i$ 
5:   else
6:      $e \leftarrow e + a_i$ 
7:   end if
8:   if  $e > best$  then
9:      $best \leftarrow e$ ;  $(L, R) \leftarrow (start, i)$ 
10:  end if
11: end for
12: return  $(best, L, R)$ 
```

例如，数组 $(-2, 11, -4, 13, -5, -2)$ 对应的 $E[i]$ 依次为 $-2, 11, 7, 20, 15, 13$ ，其中最大值 20 出现在第四项，对应子序列 $(11, -4, 13)$ 。不难分析得到时间复杂度为 $O(n)$ 。□

8.3 背包问题

8.3.1 问题描述

假设你有一个容量有限的背包，面前放着一些重量和价值各不相同的物品。你希望带走尽可能多的价值，但物品的总重量不能超过背包容量，这就是背包问题。我们先考虑每件物品只有一件、且物品必须整件装入（不可拆分）的情况，这称为 0-1 背包问题。

把物品数记为 n ，第 i 件物品的重量和价值分别记为 s_i 、 v_i ，背包容量记为 C 。下面假设重量是正整数、容量是非负整数，价值也都非负。一个简单的例子如下：假设有 4 个物品，重量分别为 $s = [2, 1, 3, 2]$ ，价值分别为 $v = [12, 10, 20, 15]$ ，背包容量为 $C = 5$ ，那么最优的装载方案是将第 1、2、4 个物品放进背包，总价值为 $12 + 10 + 15 = 37$ 。

8.3.2 最优子结构：一维似乎不再适用

仿照前面的独立集合问题，我们也可以先决定最后一件物品是否放进背包。不过，这次选了一件物品以后，除了少考虑一件物品，还会少掉一部分可用容量：

1. 不选第 n 件物品，就从前 $n - 1$ 件中选择，总重量不超过 C 。
2. 选第 n 件物品，则要先确认它放得下，即 $s_n \leq C$ 。放入以后，背包只剩下 $C - s_n$ 的容量，再从前 $n - 1$ 件中选择。

因此，只知道前 $n - 1$ 件物品的最优价值还不够，还得知道这是在多大容量下得到的。我们需要同时用物品数和可用容量来区分子问题。

令 $V_{i,c}$ 表示从前 i 件物品中选择、总重量不超过 c 时的最大价值。固定是否选第 i 件后，剩余部分必须是相应子问题的最优解；否则用更好的方案替换它，仍满足容量限制，却会提高总价值。因此

$$V_{i,c} = \begin{cases} V_{i-1,c}, & s_i > c, \\ \max\{V_{i-1,c}, V_{i-1,c-s_i} + v_i\}, & s_i \leq c. \end{cases}$$

没有物品可选时，无论容量多少，最大价值都是 0，即初值 $V_{0,c} = 0$ 。

算法 8.5 0-1 背包：求解最优值

输入： 正整数重量 $s[1..n]$ ，价值 $v[1..n]$ ，整数容量 $C \geq 0$

```

1: for  $c \leftarrow 0$  to  $C$  do
2:    $V[0, c] \leftarrow 0$ 
3: end for
4: for  $i \leftarrow 1$  to  $n$  do
5:   for  $c \leftarrow 0$  to  $C$  do
6:      $V[i, c] \leftarrow V[i - 1, c]$ 
7:     if  $s_i \leq c$  then
8:        $V[i, c] \leftarrow \max\{V[i, c], V[i - 1, c - s_i] + v_i\}$ 
9:     end if
10:  end for
11: end for
12: return  $V[n, C]$  和表  $V$ 
```

上述算法需要填写的表格共有 $(n + 1)(C + 1)$ 项，每一项只需要常数次比较和加法，所以时间、空间都是 $\Theta((n + 1)(C + 1))$ ，当 $n, C \geq 1$ 时通常简写为 $O(nC)$ 。这个复杂度有一点值得留意：这并不是多项式时间的！它取决于容量 C 的数值。注意，当输入容量 2^k 时，输入的二进制只需写 $k + 1$ 位，算法却要在每行计算 $2^k + 1$ 项。因此，运行时间虽然是 n 和 C 的多项式，却可能随输入位数指数增长。这样的算法称为**伪多项式时间算法**。

8.3.3 解的重构

从 $V[n, C]$ 开始检查。如果 $V[i, c] = V[i - 1, c]$ ，不选第 i 件也能达到最优值，就继续检查前 $i - 1$ 件。如果 $V[i, c] > V[i - 1, c]$ ，则最优值来自选中第 i 件的情况，记录这件物品，并把剩余容量减去

s_i 。两种情况都使物品下标减一，直到没有物品需要检查，因此恢复时间为 $O(n)$ 。和最大加权独立集问题类似，此时最优解可能不唯一，但只要选定一种方案即可。

算法 8.6 0-1 背包：重构最优物品选择

输入：已计算得到的二维表 V

```

1:  $c \leftarrow C$ ;  $S \leftarrow \emptyset$ 
2: for  $i \leftarrow n$  downto 1 do
3:   if  $V[i, c] > V[i - 1, c]$  then
4:     将  $i$  加入  $S$ ;  $c \leftarrow c - s_i$ 
5:   end if
6: end for
7: return  $S$ 

```

8.3.4 降维

回顾计算最优值的过程，我们会发现，在填表时，计算第 i 行只会查第 $i - 1$ 行，已经算过的更前几行便没有必要保留，因此我们只需要保留当前行和上一行的结果。事实上，我们还可以更进一步，把存储空间缩减到一个数组。下面的代码用一个数组 A 来保存当前行的结果，其中处理第 i 个物品时， $A[c]$ 表示只考虑前 $i - 1$ 件物品、容量为 c 时的最大价值。

算法 8.7 0-1 背包的一维实现

```

1:  $A[0..C] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   for  $c \leftarrow C$  downto  $s_i$  do
4:      $A[c] \leftarrow \max\{A[c], A[c - s_i] + v_i\}$ 
5:   end for
6: end for
7: return  $A[C]$ 

```

从上面的伪代码可以看出，我们是从大容量往小容量更新的，即先更新 c 比较大的 $A[c]$ 。这是因为更新 $A[c]$ 时需要 $A[c - s_i]$ ，如果从小容量往大容量更新， $A[c - s_i]$ 可能已经被更新为加入第 i 件物品后的值，从而导致选择两次物品 i ，得到错误的结果。

这样，求最优值只需 $O(C)$ 空间，时间复杂度不变。但要注意的是，如果要恢复最优解，仍然需要保存整个二维表 V ，因为一维数组只保存了当前行的结果，无法判断某个容量下的最优值是来自选中还是不选中当前物品。

8.3.5 几种背包变体

基于 0-1 背包问题，我们可以修改物品的数量或选取方式，得到下面几种变体。仍记背包容量为 C ，物品重量为 s_i ，价值为 v_i 。以下重量均为正整数，价值非负。

例题 8.2 (完全背包). 若每种物品都可以取任意多件, 怎样修改 0-1 背包的递推式? 如果只需要计算最优值, 能否使用一维数组实现?

解答. 令 $U[i, c]$ 表示只使用前 i 种物品、容量为 c 时的最大价值, 初值为 $U[0, c] = 0$. 我们可以先决定第 i 种物品取不取. 若取一件, 剩余容量减去 s_i , 但接下来还可以继续取同一种物品, 所以第二项应当使用索引 i :

$$U[i, c] = \max\{U[i-1, c], U[i, c-s_i] + v_i\} \quad (c \geq s_i).$$

因此时间复杂度仍然为 $O(nC)$. 用一维数组实现时, 从小容量往大容量更新即可, 一方面此时不限制只能选一次物品 i , 另一方面根据上述递推式, $U[i, c]$ 依赖于 $U[i, c-s_i]$, 因此必须先更新小容量的值. $\square$

例题 8.3 (有界背包). 若第 i 种物品最多只能取 b_i 件, 其中 b_i 为非负整数, 怎样求最大总价值并恢复所选物品? 当物品数量较多时, 如何转化为 0-1 背包问题求解?

解答. 令 $V[i, c]$ 表示只使用前 i 种物品、容量为 c 时的最大价值. 枚举第 i 种物品取多少件, 就有

$$V[i, c] = \max_{0 \leq k \leq \min\{b_i, \lfloor c/s_i \rfloor\}} \{V[i-1, c-ks_i] + kv_i\}.$$

初值为 $V[0, c] = 0$. 对于第 i 种物品, 要计算容量 $c = 0, 1, \dots, C$ 对应的 $C+1$ 个表项. 每个表项最多尝试 $k = 0, 1, \dots, b_i$ 这 b_i+1 种取法, 每种取法只需查表、加法和比较. 因此, 处理第 i 种物品需要 $O((C+1)(b_i+1))$ 时间, 对所有物品种类求和, 得到时间复杂度为 $O(\sum_{i=1}^n (C+1)(b_i+1)) = O((C+1)\sum_{i=1}^n (b_i+1))$.

当 b_i 很大时, 逐一尝试取 0 件、1 件、2 件等会花费很多时间. 另一种办法是把这些物品预先分组, 规定每组要么全部选入, 要么全部不选, 即把整组视为一件新的 0-1 物品. 例如, 某种物品有 13 件, 每件重量为 s_i 、价值为 v_i , 可以分成分别含 1、2、4、6 件的四组, 对应四件新物品:

$$(\text{重量}, \text{价值}) = (s_i, v_i), (2s_i, 2v_i), (4s_i, 4v_i), (6s_i, 6v_i).$$

这样, 原来“这一种物品取几件”的选择, 就变成了“这四组分别选不选”. 例如, 取 5 件可以选含 1 件和 4 件的两组; 取 9 件可以选含 1、2、6 件的三组. 关键在于不能漏掉任何取用件数: 前三组可以组合出 0 到 7 件, 再选上含 6 件的最后一组, 就能组合出 6 到 13 件. 因此, 0 到 13 件的所有取法都能表示, 重量和价值也与原来的取法相同.

对一般的 b_i , 先分出 1 件, 再分出 2 件、4 件、8 件, 依次加倍; 若剩余件数不足下一组所需的数量, 就把剩余物品组成最后一组. 为什么这样分组不会漏掉任何取用件数? 含 1、2、4、...、 2^{t-1} 件的各组可以组合出 0 到 $2^t - 1$ 件, 这就是整数的二进制表示. 最后若剩下 r 件, 则 $1 \leq r \leq 2^t$; 选上这组后, 又可以组合出 r 到 $r + 2^t - 1$ 件. 两个范围相接或重叠, 合起来正好覆盖 0 到 b_i 件.

基于上述讨论, 每种物品分出的组数为 $O(\log(b_i+1))$, 因此只需求解一个共有 $O(\sum_i \log(b_i+1))$ 件新物品的 0-1 背包, 故总时间为 $O((C+1)(1 + \sum_{i=1}^n \log(b_i+1)))$. 原先对第 i 种物品要枚举多达 b_i+1 个件数, 现在只需处理约 $\log(b_i+1)$ 件新物品, 这就是分组能够减少计算量的原因. $\square$

例题 8.4 (分数背包). 若每件物品可以切开, 取其中比例为 $x_i \in [0, 1]$ 的部分, 重量和价值也相应变为 $x_i s_i$ 、 $x_i v_i$, 怎样使装入背包的总价值最大?

解答. 很自然地, 可以直接按单位重量价值 v_i/s_i 从高到低装入物品。某件物品放不下时, 只取剩余容量能够容纳的部分, 随后停止。这样选择的理由是: 若还没有取完单位重量价值较高的物品, 却先取了较低的物品, 就可以用前者替换后者的等重部分, 在不增加总重量的情况下提高总价值。时间复杂度来源于首先按单位重量价值排序需要 $O(n \log n)$ 的时间, 随后线性时间扫描一遍即可。这是贪心算法的一个例子, 之后还会进一步讨论。 $\square$

8.3.6 两个字符串问题

背包问题用两个参数表示子问题。下面两个经典的字符串问题也需要两个参数来表示子问题。

为了描述下面的问题, 首先要给出子序列的定义: 从字符串中删去若干字符, 并保持其余字符的先后顺序, 得到的就是一个子序列。例如, `ace` 是 `abcde` 的子序列, `aec` 则不是。特别注意子序列的字符在原序列中不必连续, 如果连续则称为子串。

例题 8.5 (最长公共子序列). 给定两个字符串 $X = x_1 \cdots x_m$ 、 $Y = y_1 \cdots y_n$, 求它们的最长公共子序列 (longest common subsequence, LCS) 及其长度。例如, `abcde` 与 `ace` 的 LCS 为 `ace`, 长度为 3。

解答. 自然地, 我们可以用 $L[i, j]$ 表示 X 的前 i 个字符与 Y 的前 j 个字符的 LCS 长度。我们先比较它们的最后一个字符 x_i 和 y_j 。如果 $x_i = y_j$, 就存在一个将这两个末字符匹配的最长公共子序列。完成这次匹配后, 剩余部分是 $X[1..i-1]$ 与 $Y[1..j-1]$ 的公共子序列, 长度至多为 $L[i-1, j-1]$, 所以当前最优值恰为 $L[i-1, j-1] + 1$ 。

如果 $x_i \neq y_j$, 就至少有一个不能使用, 因此我们可以分别尝试去掉 x_i 或去掉 y_j , 再取较长的结果。综上,

$$L[i, j] = \begin{cases} 0, & i = 0 \text{ 或 } j = 0, \\ L[i-1, j-1] + 1, & i, j > 0 \text{ 且 } x_i = y_j, \\ \max\{L[i-1, j], L[i, j-1]\}, & i, j > 0 \text{ 且 } x_i \neq y_j. \end{cases}$$

不难看出, 基于上述递推式得到的算法的时间复杂度为 $O(mn)$ 。从表恢复出解的过程也是简单的: 如果 $x_i = y_j$, 就记录这个字符, 并转到 $L[i-1, j-1]$, 寻找它前面的部分。如果字符不同, 就比较 $L[i-1, j]$ 与 $L[i, j-1]$: 前者较大时跳过 x_i , 后者较大时跳过 y_j , 相等时任选其一即可, 恢复的时间复杂度为 $O(m+n)$ 。

如果只求最优值, 填表时保留当前行和上一行就够了。让较短的字符串对应列下标, 空间就能减少到 $O(\min\{m, n\} + 1)$ 。不过, 刚才恢复字符的方法还要回头查更早的行, 因此要想同时节省空间并恢复答案, 还需要另想办法。第 8.9 节会介绍这样的算法。 $\square$

例题 8.6 (序列对齐). 在两个字符串中插入空位, 使它们成为等长的两行, 并保持原有字符的顺序, 就得到一种**序列对齐**。字符对空位的代价为 $g > 0$, 字符 a, b 相对的代价为 $\alpha(a, b) \geq 0$ 。给定字符串 $X = x_1 \cdots x_m$ 、 $Y = y_1 \cdots y_n$ 及这些代价, 求总代价最小的对齐。

例如取

$$X = \text{AGGGCT}, \quad Y = \text{AGGCA}, \quad g = 1, \quad \alpha(a, b) = \begin{cases} 0, & a = b, \\ 2, & a \neq b. \end{cases}$$

例如

A G G G C T

A - G G C A

含一个空位和一次 T/A 错配，总代价为 3。请设计动态规划算法，求最小总代价并恢复一种最优对齐。

解答. 令 $P[i, j]$ 表示 $X[1..i]$ 与 $Y[1..j]$ 的最小对齐代价。最后一列有三种可能： x_i 与 y_j 对齐， x_i 与空位对齐，或者空位与 y_j 对齐。第一种情况两个字符串都继续向前考虑子问题，另外两种各有一个字符串继续向前考虑子问题。加上最后一列的代价，得到

$$P[i, j] = \min\{P[i-1, j-1] + \alpha(x_i, y_j), P[i-1, j] + g, P[i, j-1] + g\}.$$

初值为 $P[i, 0] = ig, P[0, j] = jg$ ，因为一个字符串为空时，另一串的每个字符都必须对着空位。不难看出，基于上述递推式得到的算法的时间复杂度为 $O(mn)$ 。恢复最优解的过程是简单的，此处不再赘述，时间复杂度为 $O(m+n)$ 。□

8.4 矩阵链乘法的计算顺序

8.4.1 问题描述

矩阵乘法满足结合律，但不同的加括号方式可能带来完全不同的运算量。这里固定使用普通矩阵乘法：一个 $a \times b$ 矩阵乘一个 $b \times c$ 矩阵，需要 abc 次标量乘法。给定

$$M_1 : 10 \times 20, \quad M_2 : 20 \times 50, \quad M_3 : 50 \times 1, \quad M_4 : 1 \times 100,$$

若按 $M_1(M_2(M_3M_4))$ 计算，需要

$$50 \cdot 1 \cdot 100 + 20 \cdot 50 \cdot 100 + 10 \cdot 20 \cdot 100 = 125000$$

次乘法；按 $(M_1(M_2M_3))M_4$ 计算则只要

$$20 \cdot 50 \cdot 1 + 10 \cdot 20 \cdot 1 + 10 \cdot 1 \cdot 100 = 2200.$$

同样的四个矩阵，只是改变计算顺序，乘法次数就相差这么多，因此我们希望找出使得矩阵链乘法次数最少的加括号方式。

8.4.2 最优子结构：从最后一次乘法划分矩阵链

前面的背包问题可以先决定最后一件物品取不取，再去处理前面的物品。矩阵链中，如果先把前 $n-1$ 个矩阵全部乘完，再乘 M_n ，就已经规定了最后一次乘法只能把 M_n 单独放在右边。这样会漏掉其他计算顺序，例如 $(M_1M_2)(M_3M_4)$ 。

不过，“先看最后一步”的思路仍然适用。事实上，无论括号怎样加，最后一次乘法总是把两段相邻矩阵链的结果相乘。例如对于四个矩阵，最后一步只有以下三种分法：

$$M_1 \mid M_2M_3M_4, \quad M_1M_2 \mid M_3M_4, \quad M_1M_2M_3 \mid M_4.$$

设 M_i 的大小为 $r_{i-1} \times r_i$ 。对于 $M_i \cdots M_j$, 若最后一次乘法在 M_k 后分开, 就先分别计算 $M_i \cdots M_k$ 和 $M_{k+1} \cdots M_j$, 并且最优解要求两段都应采用各自的最优顺序, 否则可以得到更优的整体顺序。令 m_{ij} 表示计算 $M_i \cdots M_j$ 最少需要的标量乘法次数。左右两段的结果矩阵分别为 $r_{i-1} \times r_k$ 和 $r_k \times r_j$, 最后一次相乘需要 $r_{i-1}r_kr_j$ 次标量乘法。因此

$$m_{ij} = \begin{cases} 0, & i = j, \\ \min_{i \leq k < j} \{m_{ik} + m_{k+1,j} + r_{i-1}r_kr_j\}, & i < j. \end{cases}$$

当 $i = j$ 时只有一个矩阵, 不需要乘法, 所以初值为 0。代码实现时我们应当按矩阵链的长度递增计算 (这与此前的情况不一样, 这是由于子问题设计决定的): 先设置长度为 1 的初值, 再依次计算长度为 2、3、直到 n 的所有区间。这样, 计算 m_{ij} 时, 拆分的左右两段都更短, 所需的值已经求出, 如图 8.2 所示。

算法 8.8 矩阵链最优计算顺序

输入: 矩阵维度数组 $r[0..n]$; $n = 0$ 时返回代价 0 与空方案

```

1: for  $i \leftarrow 1$  to  $n$  do
2:    $m[i, i] \leftarrow 0$ 
3: end for
4: for  $len \leftarrow 2$  to  $n$  do
5:   for  $i \leftarrow 1$  to  $n - len + 1$  do
6:      $j \leftarrow i + len - 1$ ;  $m[i, j] \leftarrow +\infty$ 
7:     for  $k \leftarrow i$  to  $j - 1$  do
8:        $q \leftarrow m[i, k] + m[k + 1, j] + r_{i-1}r_kr_j$ 
9:       if  $q < m[i, j]$  then
10:         $m[i, j] \leftarrow q$ ;  $K[i, j] \leftarrow k$ 
11:       end if
12:     end for
13:   end for
14: end for
15: return  $m[1, n]$  和表  $K$ 
```

其中表 $K[i, j]$ 记录使乘法次数最少的分点。由于长度为 len 的区间共有 $n - len + 1$ 个, 每个区间尝试 $len - 1$ 个分点, 因此枚举的总次数为 $\sum_{len=2}^n (n - len + 1)(len - 1) = \frac{n^3 - n}{6}$, 每次尝试只做常数运算, 所以时间为 $\Theta(n^3)$ 。表 m 和 K 各有 $O(n^2)$ 个表项, 空间为 $\Theta(n^2)$ 。

8.4.3 解的重构

表 $K[i, j]$ 已经记录了矩阵链 $M_i \cdots M_j$ 的最优分界, 因此我们可以自然地由 $K[1, n]$ 开始递归恢复完整的加括号方案。下面给出了具体的伪代码, 不难分析得到其时间复杂度为 $O(n)$, 最坏递归栈空间为 $O(n)$ 。当然我们也可以只通过 m 表来恢复方案, 但需要在每次处理 m_{ij} 时重新枚举分点, 时间复杂度为 $O(n^2)$ 。

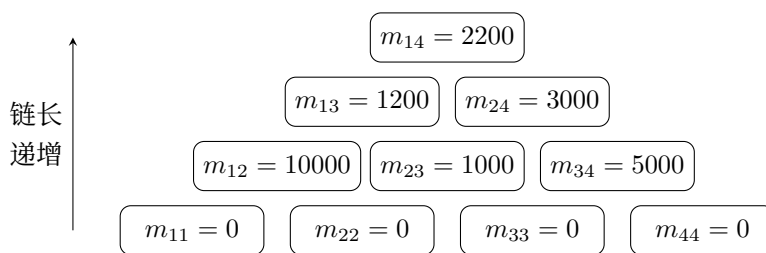


图 8.2 维度 10, 20, 50, 1, 100 的矩阵链填表顺序。先算下方短区间，再算上方长区间；最优方案的最后一次乘法在分点 $K[1, 4] = 3$ 处分开。

算法 8.9 矩阵链乘法加括号方案的恢复

```

1: procedure PRINTCHAIN( $i, j$ )
2:   if  $i > j$  then
3:     return
4:   end if
5:   if  $i = j$  then
6:     输出矩阵名  $M_i$ 
7:   else
8:     输出左括号;  $k \leftarrow K[i, j]$ 
9:     PRINTCHAIN( $i, k$ ); PRINTCHAIN( $k + 1, j$ )
10:    输出右括号
11:   end if
12: end procedure

```

8.5 最优二叉搜索树

8.5.1 问题描述

假设要用一棵二叉搜索树存储词典。不同单词被查找的频率可能相差很大，把常查的单词放得靠近根，就有机会减少查找所需的平均比较次数。那么，怎样建树才能让查找的期望比较次数最少？

把单词按字典序排为 $w_1, \dots, w_n$ ，查找它们的概率分别为 $p_1, \dots, p_n$ 。考虑每次都能查找到的情况，因此 $p_i \geq 0$ ，且 $\sum_i p_i = 1$ 。根的深度为 0，为了查找到深度为 d_i 的单词 w_i ，需要从根开始比较 $d_i + 1$ 次。因此目标是使 $\sum_{i=1}^n p_i(d_i + 1)$ 尽可能小。图 8.3 给出了一个例子。

8.5.2 算法描述

一个自然的想法是把查找概率最大的单词放在根上，或像平衡树那样尽量把左右分匀。但图 8.3 中的例子说明，这两种方法都未必最优：递归选择概率最大的单词作根，期望比较次数为 2.43；递归选择字母序在中间的单词作根，得到 2.70；右边这棵树却只需要 2.15。

回顾矩阵链乘法问题，在确定最后一次乘法的分界后，就得到了两段需要分别求解的矩阵链。建二叉搜索树也有一个类似的选择：先确定根。若选 w_k 作根，较小的单词必须全部放入左子树，较大的单词必须全部放入右子树，故接下来只需分别为这两组单词建树。由于单词已经排好序，每组仍是一段连续区间，所以这里也用两个端点表示子问题。

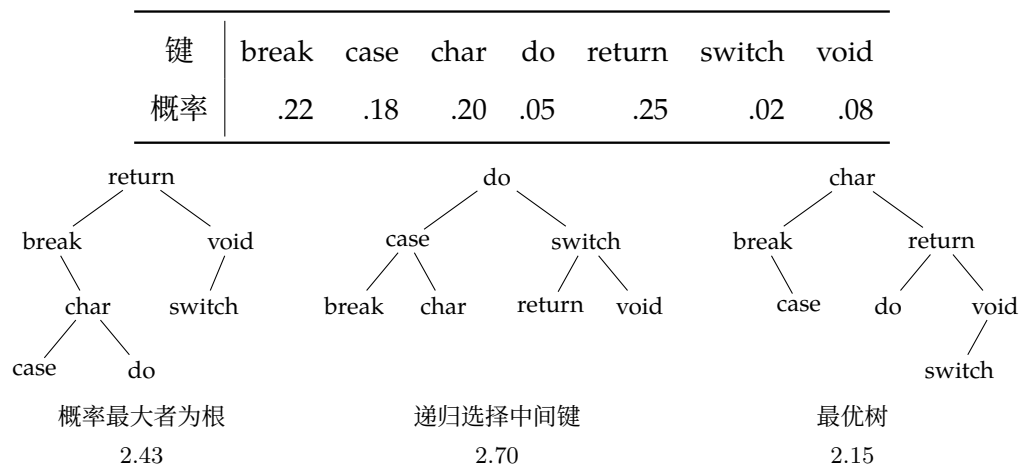


图 8.3 相同键与概率下的三棵 BST。完全平衡优化了最大深度，却不一定优化期望比较次数。

令 c_{ij} 表示为 $w_i, \dots, w_j$ 建树时，能取得的最小期望比较次数。固定 w_k 为根后，查找这些单词都要先与根比较一次，因而先产生 $\sum_{t=i}^j p_t$ 的代价。查找 w_k 至此结束，查找其他单词则要进入对应子树，左右子树中还需进行的比较分别由 $c_{i,k-1}$ 和 $c_{k+1,j}$ 给出。因此

$$c_{ij} = \sum_{t=i}^j p_t + \min_{i \leq k \leq j} \{c_{i,k-1} + c_{k+1,j}\} \quad (i \leq j).$$

这与矩阵链乘法问题的递推思路是完全类似的。在实现时，与矩阵链乘法一致，我们从最短的区间开始算起，依次计算含 1、2、直到 n 个单词的区间。一个实现的细节是，为了不重复计算区间概率和 $\sum_{t=i}^j p_t$ ，可以预先准备前缀和 $S[j] = \sum_{t=1}^j p_t$ ，以后每次用 $S[j] - S[i-1]$ 就能在常数时间内取得区间概率。

算法 8.10 最优二叉搜索树

输入： 有序互异单词表 $w[1..n]$ ，非负权重 $p[1..n]$ ；空输入返回 0 与空树

```

1:  $S[0] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $S[i] \leftarrow S[i-1] + p_i$ 
4: end for
5: for  $i \leftarrow 1$  to  $n+1$  do
6:    $c[i, i-1] \leftarrow 0$ 
7: end for
8: for  $len \leftarrow 1$  to  $n$  do
9:   for  $i \leftarrow 1$  to  $n - len + 1$  do
10:     $j \leftarrow i + len - 1$ ;  $c[i, j] \leftarrow +\infty$ 
11:    for  $k \leftarrow i$  to  $j$  do
12:       $q \leftarrow c[i, k-1] + c[k+1, j] + S[j] - S[i-1]$ 
13:      if  $q \leq c[i, j]$  then
14:         $c[i, j] \leftarrow q$ ;  $root[i, j] \leftarrow k$ 
15:      end if
16:    end for

```

```

17:   end for
18: end for
19: return  $c[1, n]$  和表  $root$ 

```

其中 $root[i, j]$ 保存一个 i, j 子区间的最优根。时间复杂度、空间复杂度以及恢复最优解的方法与矩阵链乘法问题完全类似，这里不再赘述。

8.5.3 Knuth 优化

前面的算法已经能求出最优二叉搜索树，但需要 $O(n^3)$ 时间。当单词较多时，我们希望在不影响最优解的前提下，把计算做得更快。仔细看填表的过程，每个区间都要把其中的单词逐一尝试为根，而它的两个较短区间早已计算完毕。能否利用已经求出的最优根，减少当前区间需要尝试的单词？Knuth 优化就是沿着这个思路，将时间复杂度降到 $O(n^2)$ 。

具体来说，考虑为 $w_i, \dots, w_j$ 建树。我们已经知道，去掉最后一个单词后，最优根是 $root[i, j-1]$ ；去掉第一个单词后，最优根是 $root[i+1, j]$ 。可以证明，当前区间的最优根必定落在这两个下标之间（包含端点），因此只需枚举 $root[i, j-1] \leq k \leq root[i+1, j]$ 。接下来需要说明两个问题：为什么这个范围内一定有最优根，以及缩小范围后为什么总共只需 $O(n^2)$ 时间。前一个结论要用到区间最优值之间的“四边形不等式”，我们先证明这个不等式，再用它证明最优根的范围，最后计算枚举的总次数。

为使空区间也能用同一套下标表示，下面用 $(i, j]$ 表示从 w_{i+1} 到 w_j 的单词。这样， $(i, i]$ 就是空区间， $(0, n]$ 就包含全部单词。相应地，把这一段的最优代价记为 $B[i, j]$ ，概率之和记为 $W[i, j]$ ，即 $B[i, j] = c_{i+1, j}$ ， $W[i, j] = \sum_{t=i+1}^j p_t = S[j] - S[i]$ 。于是选择 w_k 为根后，左右子问题分别是 $(i, k-1]$ 和 $(k, j]$ ，所以

$$B[i, j] = W[i, j] + \min_{i < k \leq j} \{B[i, k-1] + B[k, j]\} \quad (i < j). \quad (8.1)$$

这仍是前面的递推式，只是区间的左端点改用第一个单词的前一位下标表示。

定理 8.7 (四边形不等式). 对所有 $a \leq b \leq c \leq d$ 都有

$$W[a, c] + W[b, d] \leq W[a, d] + W[b, c], \quad W[b, c] \leq W[a, d].$$

第一个不等式称为四边形不等式：左边对应两个相交的区间 $(a, c]$ 、 $(b, d]$ ，右边对应它们的并 $(a, d]$ 与交 $(b, c]$ 。第二个不等式说，扩大区间不会减小权重和，称为区间包含单调性。由式(8.1)以及 $B[i, i] = 0$ 定义的 B 也满足四边形不等式。

在最优搜索树中， W 是区间内各单词的非负概率之和。扩大区间只会增加非负项，所以满足区间包含单调性；四边形不等式的两边则包含相同的概率项，恰好相等。因此 W 显然满足命题的条件，因此只要证明该性质能传递给 B 。

证明. 先证明 $B[a, c] + B[b, d] \leq B[a, d] + B[b, c]$ ，对 $d-a$ 作归纳。当 $d-a=0$ 时，各项均为 0。假设四个端点中最大值与最小值之差小于 $d-a$ 时，不等式成立。若 $a=b$ 或 $c=d$ ，不等式两边相等，故以下只需考虑 $a < b \leq c < d$ 的情况。设区间 $(a, d]$ 的最优树以 w_r 为根，按 r 的位置分三种情况。

若 $r \leq b$ ，则 w_r 也在区间 $(a, c]$ 中。尝试用它作根，左右子树分别取最优解，就得到 $B[a, c]$ 的一

个上界。再对端点 r, b, c, d 使用归纳假设（因为 $d - r < d - a$ ），得到

$$\begin{aligned} B[a, c] + B[b, d] &\leq W[a, c] + B[a, r - 1] + B[r, c] + B[b, d] \\ &\leq W[a, c] + B[a, r - 1] + B[r, d] + B[b, c] \\ &\leq B[a, d] + B[b, c]. \end{aligned}$$

最后一步用到 $W[a, c] \leq W[a, d]$ 。若 $r > c$ ，可以用 r 作为 $B[b, d]$ 的候选根。此时 $r - 1 < d$ ，可以对端点 $a, b, c, r - 1$ 使用归纳假设，得到

$$\begin{aligned} B[a, c] + B[b, d] &\leq B[a, c] + W[b, d] + B[b, r - 1] + B[r, d] \\ &\leq W[b, d] + B[a, r - 1] + B[b, c] + B[r, d] \\ &\leq B[a, d] + B[b, c]. \end{aligned}$$

当 $b = c$ 时，根必定满足 $r \leq b$ 或 $r > c$ ，所以这两种情况已经完成了交集为空时的证明。最后考虑 $b < r \leq c$ 。此时 $(b, c]$ 非空，取它的一个最优根 s 。已知右侧两项的最优值分别为

$$B[a, d] = W[a, d] + B[a, r - 1] + B[r, d], \quad B[b, c] = W[b, c] + B[b, s - 1] + B[s, c].$$

若 $s \leq r$ ，在左侧两个区间 $(a, c]$ 、 $(b, d]$ 中分别尝试根 s, r ，得到

$$B[a, c] + B[b, d] \leq W[a, c] + W[b, d] + B[a, s - 1] + B[s, c] + B[b, r - 1] + B[r, d].$$

把这个上界与前面展开的 $B[a, d] + B[b, c]$ 比较，两式都含有 $B[s, c]$ 和 $B[r, d]$ ，这两项可以抵消。剩余的子树代价满足

$$B[a, s - 1] + B[b, r - 1] \leq B[a, r - 1] + B[b, s - 1],$$

这是对端点 $a, b, s - 1, r - 1$ 使用归纳假设的结果。再利用 $W[a, c] + W[b, d] \leq W[a, d] + W[b, c]$ ，就得到待证不等式。

若 $s > r$ ，改用 r, s 分别作为 $(a, c]$ 、 $(b, d]$ 的候选根。把所得上界与 $B[a, d] + B[b, c]$ 比较，这次可以抵消 $B[a, r - 1]$ 和 $B[b, s - 1]$ ，余下需要证明

$$B[r, c] + B[s, d] \leq B[r, d] + B[s, c].$$

它由端点 r, s, c, d 的归纳假设得到，再加上 W 的四边形不等式即可。至此， r 和 s 的所有位置都已考虑，归纳完成。□

现在用四边形不等式证明最优根的单调性。不过，一个区间可能有多个单词都能作最优根，需要先约定选哪一个。这里统一选择其中下标最大的单词，前面的伪代码在代价相等时也更新根，正好采用了这一约定。在这一约定下，我们将证明：在区间右端增加一个单词，或者从左端删去一个单词，所选最优根的下标都不会减小。

需要注意的是，这一约定是重要的：如果每个区间都随意选择一个最优根，就未必有单调性。例如，设 $p_1 = p_2 = p_3 = 0$ ，则这些单词无论怎样建树，代价都是 0。对 $(0, 2]$ 选择 w_2 作根，对 $(0, 3]$ 却选择 w_1 作根，两次选择都最优，但增加一个单词后，根反而向左移动了。

定理 8.8 (最优根的单调性). 令 $R[i, j]$ 为区间 $(i, j]$ 中下标最大的最优根, 则对 $0 \leq i < j < n$ 有

$$R[i, j] \leq R[i, j+1] \leq R[i+1, j+1].$$

证明. 先固定左端点 i , 证明右端点从 j 增加到 $j+1$ 时, 最优根不会向左移. 对于同一个区间, W 与根的选择无关, 因此只需比较左右子树的代价和. 记这个和为 $F_j(k) = B[i, k-1] + B[k, j]$, 设旧最优根为 $u = R[i, j]$, 新最优根为 $v = R[i, j+1]$. 反设 $u > v$, 两个根就都是两个区间的合法候选. 由 B 的四边形不等式,

$$B[v, j] + B[u, j+1] \leq B[v, j+1] + B[u, j],$$

移项后, 在两边都加上 $B[i, u-1] - B[i, v-1]$, 就能将左右子树的代价合成 F , 得到

$$F_{j+1}(u) - F_{j+1}(v) \leq F_j(u) - F_j(v) \leq 0.$$

最右侧不大于 0, 是因为 u 是 $(i, j]$ 区间中的最优根. 而在新区间中, v 已经最优, 所以最左侧又不小于 0. 因此两边只能都等于 0, 说明 u 在新区间中也最优. 但 $u > v$, 这与 v 是新区间中下标最大的最优根矛盾.

再证明从左端删去一个单词时的结论. 固定右端点 j , 并设删去单词后区间仍非空. 记删除前后的最优根分别为 $u = R[i, j]$ 、 $v = R[i+1, j]$, 仍反设 $u > v$, 即最优根向左移动. 令 $G_i(k) = B[i, k-1] + B[k, j]$. 对按顺序排列的端点 $i, i+1, v-1, u-1$ 使用四边形不等式, 得到

$$B[i, v-1] + B[i+1, u-1] \leq B[i, u-1] + B[i+1, v-1].$$

移项并加上两个候选根的右子树代价之差 $B[u, j] - B[v, j]$, 就有

$$G_{i+1}(u) - G_{i+1}(v) \leq G_i(u) - G_i(v) \leq 0.$$

和前面右端点增加一个单词的情况一致, 可以得到矛盾, 完成证明. $\square$

根据上述结论, 对于至少含两个单词的区间 $(i, j]$, 从 $(i, j-1]$ 增加最后一个单词, 得到 $R[i, j-1] \leq R[i, j]$; 从 $(i, j]$ 删去第一个单词, 得到 $R[i, j] \leq R[i+1, j]$. 所以计算 $B[i, j]$ 时, 可以把枚举范围缩小为 $R[i, j-1] \leq k \leq R[i+1, j]$, 从而得到如下优化后的算法.

算法 8.11 Knuth 优化后的最优 BST

输入: 非负权重 $p[1..n]$; 先计算前缀和 S ; 空输入返回 0 与空树

```

1: for  $i \leftarrow 0$  to  $n$  do
2:    $B[i, i] \leftarrow 0$ 
3: end for
4: for  $i \leftarrow 0$  to  $n-1$  do
5:    $B[i, i+1] \leftarrow p_{i+1}$ ;  $R[i, i+1] \leftarrow i+1$ 
6: end for
7: for  $len \leftarrow 2$  to  $n$  do
8:   for  $i \leftarrow 0$  to  $n-len$  do
9:      $j \leftarrow i+len$ ;  $B[i, j] \leftarrow +\infty$ 
```

```

10:   for  $k \leftarrow R[i, j - 1]$  to  $R[i + 1, j]$  do
11:        $q \leftarrow B[i, k - 1] + B[k, j] + S[j] - S[i]$ 
12:       if  $q \leq B[i, j]$  then
13:            $B[i, j] \leftarrow q$ ;  $R[i, j] \leftarrow k$ 
14:       end if
15:   end for
16: end for
17: end for
18: return  $B[0, n]$  和根表  $R$ 

```

最后计算改进后的时间复杂度。我们把含有相同单词数的区间放在一起，统计它们总共尝试多少次根。设这些区间都含 $d \geq 2$ 个单词，区间 $(i, i + d]$ 的枚举次数为 $R[i + 1, i + d] - R[i, i + d - 1] + 1$ 。再看紧接着的区间 $(i + 1, i + d + 1]$ ，它的候选根从 $R[i + 1, i + d]$ 开始，恰好接在前一个区间的枚举终点。于是，依次把这些区间的枚举次数相加，中间的根下标一正一负相互抵消，只剩首尾两项以及每个区间贡献的 1：

$$\sum_{i=0}^{n-d} (R[i + 1, i + d] - R[i, i + d - 1] + 1) = R[n - d + 1, n] - R[0, d - 1] + n - d + 1 \leq 2n.$$

根的下标差至多为 n ，区间数也至多为 n ，这就得到了上述界。总共只有 n 种长度，因此时间复杂度为 $O(n^2)$ 。

上述讨论假设所有要查找的词都在树中，但现实情况不一定如此。对排好序的 n 个键，一共有 $n + 1$ 个可能的间隙：最小键之前、相邻键之间，以及最大键之后。记查找落在这些间隙的概率为 $q_0, \dots, q_n$ 。对键区间 $(i, j]$ ，还要考虑间隙 $i, \dots, j$ 。这些查找同样需要先与当前根比较一次，所以将权重和改为

$$W[i, j] = \sum_{t=i+1}^j p_t + \sum_{t=i}^j q_t.$$

除此之外，递推式不变。新的 W 也满足四边形不等式和区间包含单调性，因此同样可以使用 Knuth 优化实现。

8.6 最短路问题再讨论

我们此前都已经熟悉单源最短路问题，是给定一个起点 s ，求它到其余各点的最短距离。本节进一步考虑：给定一个有向带权图，怎样求出任意两个顶点之间的最短距离？这就是全源最短路问题。最直接的办法是轮流把每个顶点作为起点，各做一次单源最短路；另一种办法则是用 Floyd-Warshall 算法，通过动态规划一起计算所有点对的距离，让 $D[i, j]$ 保存从 i 到 j 的最短距离。

这一次我们允许边权为负，因此先要解决 Dijkstra 算法不再适用的问题。图 8.4 左边给出了一个例子：从 s 出发，当前到 t 、 v 的距离分别为 -2 、 1 ，Dijkstra 会先确定 t 的距离。但经过 v 再到 t ，只需 $1 - 5 = -4$ ，反而更短。可见，有负边时，当前距离最小的顶点也可能在之后得到更短的路线。

还需要注意，负边有时会使最短距离根本不存在。若一个环上所有边的权重之和为负，就称它为负环。图右边的环权重为 -2 ，每多绕一圈，总权重就减少 2。如果从起点能到这个环，绕完后又能走到目标，那么无论找到了多短的路线，都还能再绕一圈使它更短。

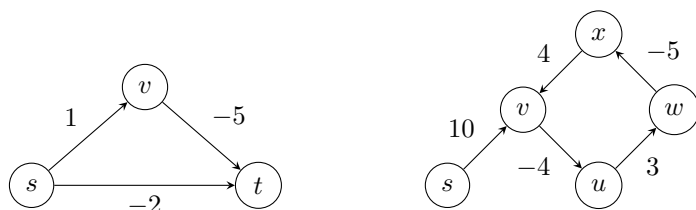


图 8.4 左：负边使 Dijkstra 过早确定 t 。右：从 s 可达的负环，环权为 -2 。

下面先介绍 Bellman–Ford 算法：给定一个源点，若从它出发到不了任何负环，就求出它到各点的最短距离；否则报告存在从源点可达的负环。再介绍 Floyd–Warshall，用动态规划一次求出所有点对的距离，并说明怎样判断哪些点对会受到负环影响。

8.6.1 Bellman–Ford 算法

Bellman–Ford 算法解决的是单源最短路问题：输入有向图和源点 s ，求从 s 到每个顶点 v 的最短距离。我们从最容易求出的情况开始：如果一步也不走，就只能停在 s ；如果最多走一条边，就检查从 s 出发的各条边；如果最多走两条边，就可以在前一步的结果后面再接一条边。这样逐步增加允许经过的边数，就能用已经算好的距离求出下一轮的距离。例如图 8.4 左边，最多走一条边时，到 v 的距离为 1，到 t 的距离为 -2 。允许走两条边后，就可以先到 v ，再沿权重为 -5 的边到达 t ，把到 t 的距离改为 $1 - 5 = -4$ 。

令 $D_k[v]$ 表示从 s 出发，至多经过 k 条边到达 v 的最小权重。如果在这个边数限制下到不了 v ，就记为 $+\infty$ 。初值为 $D_0[s] = 0$ ，其余 $D_0[v] = +\infty$ 。已知全部 D_{k-1} 后，计算 $D_k[v]$ 只需比较两类路线：一类沿用至多经过 $k-1$ 条边的旧路线；另一类先用至多 $k-1$ 条边走到某个 u ，再沿边 (u, v) 到达 v ，所以

$$D_k[v] = \min \left\{ D_{k-1}[v], \min_{(u,v) \in E} (D_{k-1}[u] + w(u, v)) \right\}. \quad (8.2)$$

我们还需要确定何时结束计算。首先说明，如果某一轮的全部距离都没有改变，就不必继续计算了。

引理 8.9 (所有距离同时稳定). 若某个 $k \geq 0$ 满足对所有 v 都有 $D_k[v] = D_{k+1}[v]$ ，则继续增加允许边数也不会改变结果，这些值就是从 s 出发的最短距离。

证明. 若第 k 轮与第 $k+1$ 轮的全部距离相同，将这些值再次代入式 (8.2)，第 $k+2$ 轮仍得到相同的结果。依此归纳，以后各轮都不会改变。□

所以算法可以在一整轮没有任何距离改变时停止。接下来考虑最多需要计算几轮。设全图共有 n 个顶点，若从 s 出发到不了负环，则计算到 D_{n-1} 就已经足够，这是因为更长的路径包含环，而没有负环的图中，包含环的路径一定不比不包含环的路径更短。因此如果第 n 轮还能减小某个距离，必定存在可达负环。

定理 8.10 (检测从源点可达的负环). 设全图共有 n 个顶点，则不存在从 s 可达的负环，当且仅当 $D_{n-1}[v] = D_n[v]$ 对所有顶点成立。

基于上述讨论，可以给出如下 Bellman–Ford 算法。其中表 P 用来记录距离是怎样得到的。若 $D[k, v]$ 直接沿用 $D[k-1, v]$ ，就记为 **carry**；若较短的路线来自边 (u, v) ，就记下这条边。这样，求出距离后便能利用 P 找回整条路线。

算法 8.12 Bellman–Ford 算法

输入： 有向图 $G = (V, E)$ 与源点 $s \in V$ ； $n = |V|$ ，边权有限

```

1:  $D[0, s] \leftarrow 0$ ；其余  $v \in V$  的  $D[0, v] \leftarrow +\infty$ 
2: for  $k \leftarrow 1$  to  $n$  do
3:   for 每个  $v \in V$  do
4:      $D[k, v] \leftarrow D[k-1, v]$ ； $P[k, v] \leftarrow \text{carry}$ 
5:   end for
6:   for 每条边  $(u, v) \in E$  do
7:     if  $D[k-1, u] < +\infty$  且  $D[k-1, u] + w(u, v) < D[k, v]$  then
8:        $D[k, v] \leftarrow D[k-1, u] + w(u, v)$ ； $P[k, v] \leftarrow (u, v)$ 
9:     end if
10:  end for
11:  if  $D[k, v] = D[k-1, v]$  对所有  $v \in V$  成立 then
12:    return 距离表  $D[k, \cdot]$ 、层数  $k$  和恢复表  $P$ 
13:  end if
14: end for
15: return “存在从  $s$  可达的负环”

```

设图有 n 个顶点、 m 条边，则每轮需要复制和比较 n 个距离，并检查 m 条边，需要 $O(n + m)$ 时间。算法最多进行 n 轮，因此有 $O(n(n + m))$ 的时间上界。

事实上，我们在实现算法的时候可以不保存完整的 D ，可以只保留各顶点当前的距离 $d[v]$ 和前驱 $pred[v]$ 。算法每轮扫描所有边，对于 (u, v) ，只要 $d[u] + w(u, v)$ 小于当前的 $d[v]$ ，就立即更新 $d[v]$ ，并将 $pred[v]$ 设为 u 。读者不难理解这一过程符合此前的递推式 (8.2)，故如下算法是合理的，并且我们可以利用 $pred$ 数组来恢复最短路径。

算法 8.13 Bellman–Ford：一维数组实现

```

1:  $d[s] \leftarrow 0$ ；其余  $d[v] \leftarrow +\infty$ ；所有  $pred[v] \leftarrow \text{NIL}$ 
2: for  $k \leftarrow 1$  to  $n - 1$  do
3:    $changed \leftarrow \text{false}$ 
4:   for 每条边  $(u, v)$  do
5:     if  $d[u] < +\infty$  且  $d[u] + w(u, v) < d[v]$  then
6:        $d[v] \leftarrow d[u] + w(u, v)$ ； $pred[v] \leftarrow u$ ； $changed \leftarrow \text{true}$ 
7:     end if
8:   end for
9:   if  $\neg changed$  then
10:    跳出外层循环

```

```

11:   end if
12: end for
13: if 存在边  $(u, v)$ ,  $d[u] < +\infty$  且  $d[u] + w(u, v) < d[v]$  then
14:   return “存在从  $s$  可达的负环”
15: end if
16: return  $(d, pred)$ 

```

Bellman-Ford 的思想也用于网络中的路由选择。数据包通常需要经过多台路由器才能到达目的地，每台路由器都要决定下一步把它交给哪个邻居。为此，路由器维护一张表，记录到各个目的地的距离估计，以及应当交给的下一台路由器。这些距离可以通过邻居提供的信息逐步算出。例如，路由器 A 到邻居 B 的代价为 1， B 告诉 A ，自己到目的地 t 的距离估计为 3，那么 A 就得到一条经 B 前往 t 、估计代价为 $1 + 3 = 4$ 的路线。如果 A 原来记录的距离为 5，就把它改为 4，并记下前往 t 的下一步应当交给 B 。对其他邻居也作同样的比较，就能选出当前估计代价最小的路线。这种方法称为距离向量方法，路由信息协议 (Routing Information Protocol, RIP) 采用的就是这一方法。²

8.6.2 Floyd-Warshall 算法

Floyd-Warshall 算法要求出任意两个顶点之间的最短距离。我们仍然从简单的情况开始：首先要求中途不经过其他顶点，然后依次允许经过顶点 $1, 2, \dots, n$ ，逐步求出全部最短距离。

先考虑没有负环的图，此时删去路线中的环不会使它更长，因此只需考虑不重复顶点的路径。记 $D_k[i, j]$ 为从 i 到 j 、中途只允许经过顶点 $1, \dots, k$ 时的最短距离。这个限制不包括起点 i 和终点 j 。初值 $D_0[i, j]$ 为直接边的最小权重，没有边时为 $+\infty$ ，且 $D_0[i, i] = 0$ 。

已知 D_{k-1} 后，再允许中途经过顶点 k 。最短路径要么不经过 k ，沿用 $D_{k-1}[i, j]$ ；要么经过 k ，此时在 k 处分成的两段都只经过编号小于 k 的内部顶点，如图 8.5 所示。因此

$$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}. \quad (8.3)$$

递推过程中逐步增大 k 的取值，最后的 D_n 就是所求的距离表。

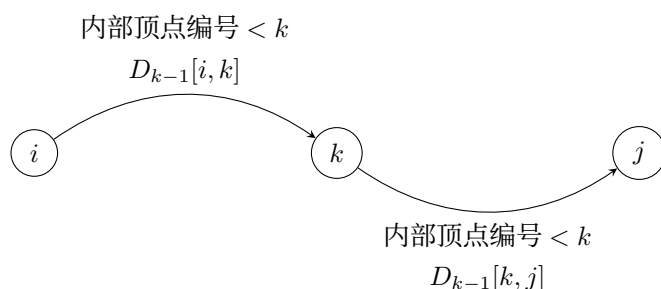


图 8.5 引入可用内部顶点 k 后，经过它的最短路径可在此分成两段。每条弧线表示一整段路径。

实现时只需保留一张二维表 D ，把 k 放在最外层循环。若经 k 得到更短的路线，就同时记下 $K[i, j] = k$ ，以便之后恢复路径。当图中有负环时，仍执行同样的更新，最后检查对角线。若 $D[k, k] < 0$ ，说明从 k 出发能绕行一圈，使总权重减小。于是，只要 i 能到 k 、 k 能到 j ，从 i 到 j 的路线就能越绕越短。代码用 $bad[i, j]$ 标记这些点对，将结果记为 $-\infty$ ，其余点对的结果仍由 $D[i, j]$ 给出。

²见 RFC 2453，第 3.4 节。

算法 8.14 Floyd–Warshall 算法

输入： 顶点编号 $1..n$ ，有限边权；允许负边、平行边和自环

```

1: 所有  $D[i, j] \leftarrow +\infty$ ,  $K[i, j] \leftarrow \text{NIL}$ ; 所有  $D[i, i] \leftarrow 0$ 
2: for 每条边  $(u, v)$  do
3:    $D[u, v] \leftarrow \min\{D[u, v], w(u, v)\}$ 
4: end for
5: for  $k \leftarrow 1$  to  $n$  do
6:   for  $i \leftarrow 1$  to  $n$  do
7:     for  $j \leftarrow 1$  to  $n$  do
8:       if  $D[i, k] < +\infty$  且  $D[k, j] < +\infty$  且  $D[i, k] + D[k, j] < D[i, j]$  then
9:          $D[i, j] \leftarrow D[i, k] + D[k, j]$ ;  $K[i, j] \leftarrow k$ 
10:      end if
11:    end for
12:  end for
13: end for
14:  $Z \leftarrow \{k : D[k, k] < 0\}$ 
15:  $\text{bad}[i, j] \leftarrow \text{false}$ , 对所有点对初始化
16: for  $k \in Z$  do
17:   for 满足  $D[i, k] < +\infty$  的  $i$  和  $D[k, j] < +\infty$  的  $j$  do
18:      $\text{bad}[i, j] \leftarrow \text{true}$ 
19:   end for
20: end for
21: return  $(D, K, \text{bad})$ ;  $\text{bad}[i, j]$  为真时将该点对的答案解释为  $-\infty$ 

```

恢复路径只处理答案有限的点对。若 $K[i, j] = k$ ，就分别恢复 i 到 k 、 k 到 j 的路径，再接起来即可。若 i, j 之间没有分点， $i \neq j$ 时使用两者之间权重最小的边即可， $i = j$ 时只输出该顶点。

最后分析复杂度。初始化读入 m 条边，三层循环检查 n^3 个组合，标记负环影响最多也需 $O(n^3)$ 时间，因此总时间为 $O(n^3 + m)$ 。

当然读者可能会有一个自然的问题：事实上，我们还可以直接对每个源点执行一次 Bellman–Ford 算法，得到所有点对的距离。那为什么还需要使用 Floyd–Warshall？下面给出两种全源最短路方法的时间比较。如果对每个源点分别执行一次 Bellman–Ford 的原地版本，总时间为 $O(n^2m + n^2)$ ，Floyd–Warshall 则为 $O(n^3 + m)$ 。因此，对 $m = \Theta(n^2)$ 的稠密图，后者将时间上界从 $O(n^4)$ 降到了 $O(n^3)$ 。

如果边权非负，还可以对每个源点执行 Dijkstra。配合第 5 讲的 Fibonacci 堆，总时间为 $O(nm + n^2 \log n)$ 。对 $m = O(n)$ 的稀疏图，这就是 $O(n^2 \log n)$ ，比 Floyd–Warshall 更快。因此，选择全源最短路算法时，需要同时考虑边权是否非负和图中边的多少。

事实上，最短路问题时至今日仍有研究。如果边权非负，使用 Dijkstra 算法配合 Fibonacci 堆的时间复杂度为 $O(m + n \log n)$ 。它每次确定距离最近的未处理顶点，因此不仅求出了各点的距离，还顺带按距离给顶点排好了序。但单源最短路问题本身只要求距离，未必需要这个顺序。这就提出了一个进一步的问题：省去全局排序，能否更快地求出这些距离？Duan、Mao、Mao、Shu、Yin 在 2025

年的工作中给出了确定性的 $O(m \log^{2/3} n)$ 算法。对 $m = \Theta(n)$ 的稀疏图，这比 $O(n \log n)$ 小，说明逐个确定全局最近点并非解决该问题的唯一方式。这一结果研究的是非负边权的单源最短路，完整算法见参考资料中的论文，这篇论文也获得了 2025 年理论计算机顶级会议 STOC 的最佳论文奖。

8.7 红黑树计数

给定 N 个互不相同的键，能建出多少棵不同的红黑树？其中只要树的形状或结点颜色不同，就算不同的方案。注意对于一种确定的树形，按中序顺序填入排好序的键，便唯一确定了各结点的键值，因此需要枚举的是树形和染色。

每棵树仍须满足红黑树的约束：根为黑色，NIL 叶子为黑色，红结点没有红孩子；从同一个结点出发到任意 NIL 叶子的路径，黑色结点数相同。一棵树可以由根和左右两棵子树组成，所以我们尝试先统计较小的子树，再把它们接起来。不过，仅知道左右各有多少个结点还不够：两棵子树从根到 NIL 的路径上，黑结点数必须相同；如果要接在一个红根下面，它们的根还必须为黑色。我们因而按结点数、路径上的黑结点数以及根颜色分别计数。

我们用 h 记录从子树根到 NIL 的路径上有多少个黑色内部结点，其中包括根但不包括 NIL。令 $B[n, h]$ 、 $R[n, h]$ 分别表示有 n 个内部结点、路径上的黑结点数为 h 、根为黑色或红色的子树数量。注意这里允许红根，是为了统计可以接到更大树中的子树。我们不难得到初值为 $B[0, 0] = 1$ ， $B[0, h] = 0 (h > 0)$ ， $R[0, h] = 0 (h \geq 0)$ 。

接下来考虑递推关系。考虑有 n 个内部结点的非空树，其中根占去一个结点，若左子树有 a 个结点，则右子树有 $n - 1 - a$ 个结点。枚举 $a = 0, \dots, n - 1$ ，便包含了所有左右结点分配。若根为黑色，它已经贡献一个黑色结点，所以左右子树的参数都应为 $h - 1$ ，子树根可以红也可以黑。若根为红色，它不改变黑色结点数，所以两棵子树的参数仍为 h ，但子树根必须为黑色。在满足颜色和黑结点数要求后，每一种左子树都能与每一种右子树搭配，因此固定 a 时要将两边的方案数相乘。最后对所有 a 求和：

$$B[n, h] = \sum_{a=0}^{n-1} (B[a, h-1] + R[a, h-1])(B[n-1-a, h-1] + R[n-1-a, h-1]),$$

$$R[n, h] = \sum_{a=0}^{n-1} B[a, h]B[n-1-a, h].$$

具体实现时，我们要给表格分配空间，还需确定 h 的上限。根据定理 2.3 的证明过程，我们有 $h \leq \lfloor \log_2(N+1) \rfloor := H$ 。

算法 8.15 红黑树计数

输入：整数 $N \geq 0$

```

1:  $H \leftarrow \lfloor \log_2(N+1) \rfloor$ ;  $B[0..N, 0..H] \leftarrow 0$ ;  $R[0..N, 0..H] \leftarrow 0$ 
2:  $B[0, 0] \leftarrow 1$ 
3: for  $n \leftarrow 1$  to  $N$  do
4:   for  $h \leftarrow 0$  to  $H$  do
5:     for  $a \leftarrow 0$  to  $n-1$  do
6:        $b \leftarrow n-1-a$ 
```

```

7:       $R[n, h] \leftarrow R[n, h] + B[a, h]B[b, h]$ 
8:      if  $h \geq 1$  then
9:           $B[n, h] \leftarrow B[n, h] + (B[a, h-1] + R[a, h-1])(B[b, h-1] + R[b, h-1])$ 
10:      end if
11:  end for
12: end for
13: end for
14: return  $\sum_{h=0}^H B[N, h]$ 

```

结点数至多为 N ，而参数 $h \leq \lfloor \log_2(N+1) \rfloor$ ，所以表中共有 $O(N \log(N+1))$ 个状态。每个状态至多尝试 N 种左右结点数分配，每次做常数级加乘，因此共需 $O(N^2 \log(N+1))$ 次算术运算。

8.8 总结

事实上，前面的许多问题（以及之后要讨论的很多问题）都要求在约束下求一个最大值或最小值，这也是运筹学研究的重要内容。例如，0-1 背包可以写成下面的整数规划模型：

$$\begin{aligned}
 & \max \sum_{i=1}^n v_i x_i, \\
 & \text{s.t. } \sum_{i=1}^n s_i x_i \leq C, \\
 & x_i \in \{0, 1\}.
 \end{aligned}$$

其中 x_i 表示第 i 件物品选不选，第一行希望总价值最大，后两行则规定不能超重、每件物品只能取 0 或 1 件。这样便把问题要求转化成了数学模型。动态规划进一步利用了物品的顺序和整数容量，给出了求解这个模型的具体算法。许多最优化问题都能用数学规划表示，但怎样高效求解，仍取决于问题本身有哪些结构可用。如果没有结构可用，则只能使用通用方法，例如整数规划求解器、分支定界法等，这将无法保证在多项式时间内得到结果；或者尝试近似算法，但无法保证得到最优解。

需要注意的是，分治和动态规划都会利用子问题的答案，区别要看这些子问题是否会被反复求解。归并排序把数组分成左右两半，两次递归分别处理不同的区间；矩阵链乘法却要尝试不同分界，同一段矩阵链的最优值会在多种选择中被反复用到。因此动态规划需要使用记忆化的方式保存子问题的答案，以避免重复计算。

设计动态规划时，可以先确定一次选择，看看还剩下什么问题。例如，在背包中选定一件物品后，剩余容量会改变；矩阵链确定最后一次乘法后，需要分别计算左右两段；红黑树计数则要求左右子树满足相同的黑结点数。这些剩余问题需要知道哪些信息，状态就应记录哪些信息。总而言之，最优子结构的确定是一个技巧性与经验性较强的过程，相信读者在见多了各种问题后，会逐渐掌握其中的规律。

8.9 补充材料：Hirschberg 算法

给定长度为 m, n 的字符串 X, Y ，我们能用 $O(mn)$ 的表求出一个 LCS。如果只要长度，每次保留相邻两行就够了，但要沿表找回具体字符，就会用到已经丢掉的行。一个自然的问题是，如何在

保存整张表的情况下找出一个具体的 LCS? 这就引入了 Hirschberg 算法。

Hirschberg 算法先将 X 分成 $X[1..r]$ 和 $X[r+1..m]$, 其中 $r = \lfloor m/2 \rfloor$ 。一个公共子序列中, 在 X 前半段选出的字符一定排在后半段字符之前, 这些字符在 Y 中也按相同顺序出现。因此, 我们可以在 Y 中选一个分界 j , 让前一部分在 $X[1..r], Y[1..j]$ 中匹配, 后一部分在 $X[r+1..m], Y[j+1..n]$ 中匹配。要找到最好的分界, 就比较 $j = 0, \dots, n$: 对每个 j , 分别求这两对字符串的 LCS 长度, 再相加。记

$$L[j] = \text{LCSLen}(X[1..r], Y[1..j]), \quad R[j] = \text{LCSLen}(X[r+1..m], Y[j+1..n]),$$

其中 LCSLen 表示 LCS 长度。分界为 j 时, 两段一共能选出 $L[j] + R[j]$ 个字符, 因此

$$\text{LCSLen}(X, Y) = \max_{0 \leq j \leq n} \{L[j] + R[j]\}.$$

选择使和最大的 j 后, 分别递归求出这两对子串的 LCS, 再按先后顺序连接, 就得到原问题的一个 LCS。

接下来考虑如何计算所有 $L[j]$ 和 $R[j]$ 。回顾正文中的 LCS 表, 第 i 行、第 j 列保存的是两个字符串的前 i 个字符与前 j 个字符的 LCS 长度。因此, 用 $X[1..r]$ 与整个 Y 填表, 算到第 r 行时, 该行的第 j 项就是 $L[j]$, 一次填表便能得到全部 $L[j]$ 。计算每一行时只会用到上一行和当前行, 所以保留两个数组即可。

$R[j]$ 对应的是 X 的后半段与 Y 的后 $n - j$ 个字符, 而刚才的表计算的是前若干个字符的答案。我们可以将 $X[r+1..m]$ 与整个 Y 同时反转, 再使用相同的递推式。反转后, Y 的后 $n - j$ 个字符变成前 $n - j$ 个字符, 公共子序列的长度则保持不变。记这次填表的最后一行为 Q , 就有 $R[j] = Q[n - j]$ 。

下面给出具体实现。先将填表过程写成函数 LastRow , 其中用 P 保存上一行, C 保存当前行, 每算完一行便交换两个数组。计算结束后返回最后一行 P , 其中 $P[t]$ 就是 U 与 $V[1..t]$ 的 LCS 长度。

算法 8.16 返回 LCS 表的最后一行

```

1: function LastRow( $U[1..a], V[1..b]$ )
2:    $P[0..b] \leftarrow 0$ , 分配  $C[0..b]$ 
3:   for  $i = 1, \dots, a$  do
4:      $C[0] \leftarrow 0$ 
5:     for  $j = 1, \dots, b$  do
6:       if  $U[i] = V[j]$  then
7:          $C[j] \leftarrow P[j - 1] + 1$ 
8:       else
9:          $C[j] \leftarrow \max(P[j], C[j - 1])$ 
10:      end if
11:    end for
12:    交换  $P, C$  的引用, 使  $P$  保存刚算完的一行,  $C$  用于计算下一行
13:  end for
14:  释放  $C$ ; return  $P$ 
15: end function

```

有了 LASTROW, 就可以按前面的思路找到分界并递归求解, 代码中使用 rev 表示数组反转。

算法 8.17 用分治恢复一个 LCS

输入: 字符串 X, Y ; 开始计算前, 建立一个空列表, 用于依次保存找到的 LCS 字符

```

1: function HIRSCHBERG( $X[1..m], Y[1..n]$ )
2:   if  $m = 0$  或  $n = 0$  then
3:     return
4:   end if
5:   if  $m = 1$  then
6:     从左到右寻找  $Y$  中第一个等于  $X[1]$  的字符
7:     若找到, 向输出列表追加  $X[1]$ ; 随后 return
8:   end if
9:    $r \leftarrow \lfloor m/2 \rfloor$ 
10:   $L \leftarrow \text{LASTROW}(X[1..r], Y)$ 
11:   $Q \leftarrow \text{LASTROW}(\text{rev}(X[r+1..m]), \text{rev}(Y))$ 
12:   $j \leftarrow$  使  $L[j] + Q[n-j]$  最大的最小下标,  $0 \leq j \leq n$ 
13:  释放  $L, Q$ 
14:  HIRSCHBERG( $X[1..r], Y[1..j]$ )
15:  HIRSCHBERG( $X[r+1..m], Y[j+1..n]$ )
16: end function

```

接下来分析时间与空间复杂度。一次调用中, 两次填表分别处理 rn 和 $(m-r)n$ 个位置, 共需 $O(mn)$ 时间; 遍历 j 找到最佳切分位置另需 $O(n)$ 时间, 因此除递归以外的时间为 $O(mn)$ 。两次递归处理的字符串长度分别为 (r, j) 和 $(m-r, n-j)$, 所以两次调用需要填的表格位置数之和至多为 $rj + (m-r)(n-j) \leq \lceil m/2 \rceil n \leq \frac{2}{3}mn$ 。依此类推, 时间复杂度为 $O(mn \sum_{k \geq 0} (2/3)^k) = O(3mn) = O(mn)$ 。

对于空间复杂度, 注意到每次填表只保留常数个长度至多为 $n+1$ 的数组, 并且在递归之前空间释放, 因此这些数组合计只占 $O(n)$ 空间。递归栈还需保存各层的子串下标和切分位置, 每层占常数空间; X 每次减半, 递归深度为 $O(\log m)$, 所以辅助空间为 $O(n + \log m)$ 。再计入输入字符串和输出的 LCS, 总空间为 $O(m + n)$ 。

事实上, 序列对齐也可以使用 Hirschberg 算法恢复最优解, 思想是完全类似的, 故不再赘述, 读者可自行尝试。

8.10 习题

习题 8.1 (一般图的加权独立集合). 考虑一个任意的无向图 $G = (V, E)$, 任意一个顶点 $v \in V$ 具有非负权重 w_v 。先令 $H = G - v$, 再从 H 中删除 v 在 G 中的所有邻点以及关联边, 得到 $K = G - N[v]$, 如下图所示:

设 W_G 、 W_H 和 W_K 分别表示 G 、 H 和 K 的最大加权独立集合的总权重, 并考虑下面这个公式:

$$W_G = \max\{W_H, W_K + w_v\},$$

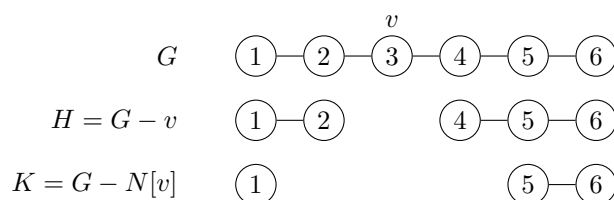


图 8.6 删去 $v = 3$ ，以及进一步删去它的邻点 2、4。结点编号在三行中保持不变。

下面哪些说法是正确的（选择所有正确的答案）：

1. 这个公式对于路径图并不总是正确的；
2. 这个公式对于路径图总是正确的，但对于树并不总是正确的；
3. 这个公式对于树总是正确的，但对于任意的图并不总是正确的；
4. 这个公式对于任意的图都是正确的；
5. 这个公式对于树的加权独立集合问题可以产生一种线性时间的算法；
6. 这个公式对于任意图的加权独立集合问题可以产生一个线性时间的算法。

习题 8.2 (零钱兑换). 假如你有若干种面额的硬币，每种都可以使用任意多枚，要凑出金额 amount ，最少需要多少枚硬币？面额由正整数数组 coins 给出，目标金额为非负整数。请同时给出一种使用最少硬币的兑换方式；若无法凑出，就返回 -1 。

例如： $\text{coins} = [1, 2, 5]$ ， $\text{amount} = 11$ ，则答案为 3，最优兑换为 $11 = 5 + 5 + 1$ 。

习题 8.3 (Decode). 把字母 $A, \dots, Z$ 分别编码为十进制串 $0, \dots, 25$ 。现在给你一个非空数字串，请用动态规划算法计算有多少种解码方式。注意单独的 0 表示一个字母，而两位编码不能带前导零，所以 07 只能拆成 0|7。

习题 8.4 (Maximum Subsequence Product). 给定非空实数数组，给出求非空连续子序列的最大乘积的线性时间算法。元素可能为负数、0 或绝对值小于 1 的数。例如 $(2, 3, 0.5, -2, 4)$ 的答案为 6。

习题 8.5 (生产线调度与历史依赖). 一件产品要依次完成 $n \geq 1$ 个加工阶段，每个阶段可在编号为 0、1 的任一生产线上完成。图 8.7 给出了一个例子。从生产线 ℓ 的第 $j-1$ 阶段出发，留在原生产线完成第 j 阶段需时 $a_{\ell,j}$ ；换到另一条生产线完成则需时 $b_{\ell,j}$ ，这些时间均非负。

1. 设计 $O(n)$ 时间的算法，求最少总用时及一条最优路线，并求出下图的结果。
2. 若 $n \geq 2$ ，并规定“第一阶段在生产线 0 加工的产品，最后一步不能从生产线 0 换到生产线 1”，算法如何修改？

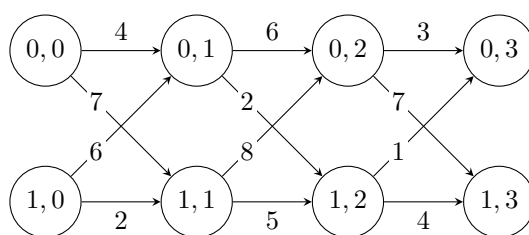


图 8.7 三阶段生产线示例。

习题 8.6 (Hand-made Cream). 假如你是一位面包师, 准备从现有的 n 片面包和 m 份奶油中挑选材料, 制作奶油面包。每片面包和每份奶油都有一个已知的口味评分, 分别记为 $a[1..n]$ 和 $b[1..m]$ 。要求一片面包搭配一份奶油, 成品的得分就是两者评分的乘积。注意评分可以为负, 所以两个负评分也可能搭配出正分。

面包和奶油各自按给定顺序排列, 你可以跳过其中一些材料, 但每份材料至多使用一次, 且选出的面包与奶油数量必须相同。配对时, 第一片选中的面包搭配第一份选中的奶油, 第二片搭配第二份, 依此类推, 不能改变顺序。例如, $a = (-1, 10, 8)$ 、 $b = (10, 8, 11, 9)$ 时, 可以选第 2、3 片面包和第 1、3 份奶油, 总得分为 $10 \cdot 10 + 8 \cdot 11 = 188$, 这也是该例的最大总分。

请设计动态规划算法, 求最大总得分及一种材料选择方案。允许一对都不选, 此时总分为 0。

习题 8.7 (回文子串与回文分割). 若一个字符串反转后与原串相同, 就称它为回文串。给定字符串 s , 设计动态规划算法解决下面两问, 并说明如何恢复答案及时间复杂度:

1. 求最长的回文子串。例如 babad 的答案可以是 bab 或 aba , 注意子串必须是在原序列中连续出现的子序列。
2. 把整个字符串分成若干回文子串, 求最少切割次数及一种分割方案。例如 aab 可分为 aa|b , 只需一刀。

习题 8.8 (最长递增子序列). 给定数组 $a[1..n]$, 按原顺序选出尽可能多的元素, 使它们严格递增, 这就是最长递增子序列 (LIS)。例如 $(3, 1, 2, 2, 4)$ 的一个答案为 $(1, 2, 4)$, 注意相等元素不能同时出现在严格递增序列中。请设计 $O(n^2)$ 时间的动态规划算法, 求最长长度并恢复一个子序列。

习题 8.9 (钢条切割). Serling 公司购买长钢条, 将其切割为短钢条出售, 公司管理层希望知道最佳的切割方案。假定已知 Serling 公司出售一段长度为 i 英寸的钢条的价格为 p_i ($i = 1, 2, \dots, n$), 单位为美元。钢条的长度均为整英寸。下图给出了一个价格表的样例。

长度 i	1	2	3	4	5	6	7	8	9	10
价格 p_i	1	5	8	9	10	17	17	20	24	30

形式化而言, 给定一段长度为 n 英寸的钢条和一个价格表 $p_i (i = 1, 2, \dots, n)$, 求切割钢条方案, 使得销售收益 r_n 最大。

习题 8.10 (Manager of Tasks). 一台机器要从时刻 0 开始, 按顺序处理任务 $1, \dots, n$ 。你可以将这些任务分成若干组, 每组包含一段连续的任务, 所有任务都必须处理。机器每处理一组, 都先花 S 时间启动, 再依次执行组内任务, 任务 i 的执行时间为 T_i 。上一组结束后, 才开始下一组的启动与处理。注意已经执行完的任务仍要留在机器中等待, 直到同组的全部任务执行完, 才一起算作完成, 任务 i 的成本为它的完成时刻乘以给定系数 C_i 。

例如, $S = 1$ 、 $T = (1, 3, 4, 2, 1)$ 、 $C = (3, 2, 3, 3, 4)$ 。若分为 $\{1, 2\}$ 、 $\{3\}$ 、 $\{4, 5\}$ 三组, 第一组在时刻 $1 + 1 + 3 = 5$ 完成, 第二组在时刻 $5 + 1 + 4 = 10$ 完成, 第三组在时刻 $10 + 1 + 2 + 1 = 14$ 完成。因此总成本为 $5(3 + 2) + 10 \cdot 3 + 14(3 + 4) = 153$ 。

请设计 $O(n^2)$ 时间的动态规划算法, 求最小总成本, 并恢复一种最优分组。

习题 8.11 (完全平方数的和). 把正整数 n 写成若干个正整数的平方之和, 最少需要几项? 例如 $13 = 4 + 9$, 需要两项。请给出动态规划算法及复杂度。

习题 8.12 (RNA 配对). 一条 RNA 链可以用由 A, C, G, U 四种字符组成的序列 $s[1..n]$ 表示, 每个字符代表一个碱基。链上的某些碱基可以两两配对, 使链折叠。这里采用一个简化模型, 要在以下规则下尽可能多地配对:

1. 只允许 A 与 U 配对、 C 与 G 配对, 每个位置至多参与一对。
2. 配对的两个位置之间至少隔着 4 个位置, 即 (i, j) 配对时须满足 $j - i \geq 5$, 其中 $i < j$ 。
3. 将序列从左到右排开, 在上方用弧线连接配对的位置, 弧线不能交叉。也就是说, 不能同时选择 (i, j) 和 (k, ℓ) , 使得 $i < k < j < \ell$ 。

例如, 对于 ACCGGUAGU, 可以将第 1、9 个位置的 A 与 U 配对, 将第 3、8 个位置的 C 与 G 配对, 符合全部规则。若改选 $(1, 6)$ 和 $(2, 8)$, 虽然字符和间隔都符合要求, 但因 $1 < 2 < 6 < 8$, 两条弧线会交叉, 所以不能同时选择。

请设计动态规划算法, 求给定序列的最大配对数, 并恢复一种配对方案, 并说明时间复杂度。

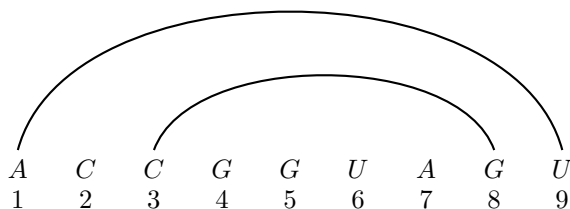


图 8.8 序列 ACCGGUAGU 的两对合法配对。

习题 8.13 (最短路问题). 判断并解释:

1. 用一维数组实现的 Bellman-Ford 算法, 在第 k 轮后恰好得到至多 k 条边的最短距离。
2. Floyd-Warshall 算法的三层循环可以任意交换。

8.11 参考答案

习题 8.1. 第 4、5 项正确。首先递推式显然对任意图成立。但在一般图中, 每个子问题要求的是剩余顶点构成的子图的最优解, 在图没有特殊结构的时候, 最多可能需要考虑 n 个顶点共 2^n 个子集, 不一定能保证得到线性时间算法。

对树可以只为每个结点保存两项: 选 u 时以其为根的子树的最大权重和为 $I[u]$, 不选 u 时的最大权重和为 $O[u]$, 则有

$$I[u] = w_u + \sum_{v \text{ 是 } u \text{ 的孩子}} O[v], \quad O[u] = \sum_{v \text{ 是 } u \text{ 的孩子}} \max\{I[v], O[v]\}.$$

叶子取 $I[u] = w_u, O[u] = 0$, 最后取根的两项中的较大者即可, 时间复杂度是 $O(n)$ 。□

习题 8.2. 记目标金额为 A , 令 $D[x]$ 表示凑出金额 x 的最少硬币数。若最后一枚的面额为 c , 其余硬币就要凑出 $x - c$, 所以

$$D[0] = 0, \quad D[x] = 1 + \min_{c \in \text{coins}, c \leq x} D[x - c].$$

共有 A 个正金额, 每个检查 q 种面额, 连同初始化需要 $O(Aq + A)$ 时间。□

习题 8.3. 令 $D[i]$ 表示前 i 位的解码方式数。最后一位可以单独表示一个字母，前面的每种解码都可接上它，因此有 $D[i-1]$ 种；若最后两位在 10 到 25 之间，还可以一起表示一个字母，再增加 $D[i-2]$ 种，由此得到时间复杂度为 $O(n)$ 的算法。 $\square$

习题 8.4. 令 $H[i], L[i]$ 为以位置 i 结尾的最大、最小乘积，二者都从 a_1 初始化。对 $i > 1$ ，候选集合为 $\{a_i, H[i-1]a_i, L[i-1]a_i\}$ ，令 $H[i], L[i]$ 分别取这三个数的最大值和最小值即可。 $\square$

习题 8.5. 第一问，令 $F[\ell, j]$ 表示完成第 j 阶段、位于生产线 ℓ 的最少总用时。最后一步只有留在原生产线或从另一条生产线换来两种可能，因此

$$F[\ell, j] = \min\{F[\ell, j-1] + a_{\ell, j}, F[1-\ell, j-1] + b_{1-\ell, j}\}.$$

初值为 $F[0, 0] = F[1, 0] = 0$ ，最后选择 $F[0, n], F[1, n]$ 中较小的一项即可。图中一条最优路线是 $0 \rightarrow 0 \rightarrow 1 \rightarrow 0$ ，总用时为 7。

第二问中，同样到达生产线 ℓ 的第 j 阶段，若第一阶段所在的生产线不同，最后一次换生产线受到的限制也可能不同，因此状态还要记录第一阶段的生产线编号 z 。令 $F[\ell, j, z]$ 表示第一阶段在生产线 z 加工、完成第 j 阶段后位于生产线 ℓ 的最少用时。初始化为

$$F[z, 1, z] = \min\{a_{z, 1}, b_{1-z, 1}\}, \quad F[1-z, 1, z] = +\infty \quad (z = 0, 1).$$

从第 2 阶段开始，对每个 z 分别使用第一问的递推；仅在 $j = n, z = 0, \ell = 1$ 时，删去从生产线 0 换来的候选，只能沿生产线 1 继续加工。答案为所有 $F[\ell, n, z]$ 的最小值，时间复杂度仍为 $O(n)$ 。 $\square$

习题 8.6. 解决思路与 LCS 非常类似，先考虑两个序列的末项怎样处理。若 a_i, b_j 都被选中，由于配对必须保持先后顺序，它们就必须配成最后一对；否则，至少要跳过其中一个。用 $F[i, j]$ 表示前 i 个面包评分与前 j 个奶油评分能得到的最大总分，就有

$$F[i, j] = \max\{F[i-1, j], F[i, j-1], F[i-1, j-1] + a_i b_j\}.$$

其中一个序列为空时无法配对，所以初值为 $F[0, j] = F[i, 0] = 0$ 。填表时间复杂度为 $O(nm + n + m)$ ，恢复与 LCS 类似，需要 $O(n + m)$ 时间。 $\square$

习题 8.7. 第一问，一段连续子串是否回文，由两端字符以及去掉两端后的中间部分决定。令 $H[i, j]$ 表示 $s[i..j]$ 是否回文，则

$$H[i, j] = (s_i = s_j) \wedge (j - i \leq 1 \text{ 或 } H[i+1, j-1]).$$

单个字符总是回文，两个字符则只需相同。更长的子串要求两端相同，且中间仍为回文。实现时按子串长度递增填表即可，共有 $O(n^2)$ 个区间，每个区间只需常数时间判断，所以时间复杂度为 $O(n^2)$ 。恢复子串的操作是简单的，不再赘述。

第二问可以继续使用回文表 H 。令 $F[j]$ 为前 j 个字符最少分成的段数。最后一段若为 $s[i..j]$ ，前面需要 $F[i-1]$ 段，所以

$$F[0] = 0, \quad F[j] = \min_{1 \leq i \leq j, H[i, j]} \{F[i-1] + 1\}.$$

$\square$

习题 8.8. 可以先固定最后选中的元素 a_i , 用 $E[i]$ 表示以它结尾的最长递增子序列长度。如果前面还选了元素, 倒数第二项就必须来自某个 $j < i$, 而且满足 $a_j < a_i$ 。在所有这样的 j 中, 选择 $E[j]$ 最大的一个, 再接上 a_i , 就得到

$$E[i] = 1 + \max(\{E[j] : j < i, a_j < a_i\} \cup \{0\}).$$

其中的 0 对应前面没有合适元素, 只选 a_i 自己的情形。基于这一递推式, 不难得到 $O(n^2)$ 的算法。为了找回选出的元素, 计算 $E[i]$ 时还用 $prev[i]$ 记下所选的倒数第二项的下标。最后从长度最大的那一项出发, 沿这些前驱下标往前走, 就能把子序列恢复出来。□

习题 8.9. 可以先决定卖出的第一段有多长。若一条长 j 的钢条先取出长 i 的一段, 这一段能卖 p_i , 剩下的长 $j - i$ 的钢条还要尽可能卖得贵。因此, 用 $R[j]$ 表示长 j 的钢条能够得到的最高收益, 就有

$$R[0] = 0, \quad R[j] = \max_{1 \leq i \leq j} \{p_i + R[j - i]\}.$$

□

习题 8.10. 令 $F[i]$ 表示任务 $i, \dots, n$ 从时刻 0 开始处理时, 能够取得的最小总成本, 其中没有任务时取 $F[n + 1] = 0$ 。假如第一组包含任务 $i, \dots, j$, 那么完成这一组需要 $d = S + \sum_{k=i}^j T_k$ 时间。第一组中的任务都在时刻 d 完成, 因此成本为 $d \sum_{k=i}^j C_k$ 。剩余任务如果从时刻 0 开始, 最小成本是 $F[j + 1]$, 但它们必须等第一组结束才能开始, 所有完成时刻都往后推迟了 d , 就还要增加 $d \sum_{k=j+1}^n C_k$ 的成本, 因此

$$F[i] = \min_{i \leq j \leq n} \left\{ F[j + 1] + \left(S + \sum_{k=i}^j T_k \right) \sum_{k=i}^j C_k \right\}.$$

基于这一递推式, 我们不难得到 $O(n^2)$ 的算法。为了恢复分组方案, 可以同步记录每个 $F[i]$ 对应的一个最优分点。□

习题 8.11. 把每个平方数看成一种硬币面额, 这道题就变成了习题 8.2 中的零钱兑换问题。能够用到的面额为 $1^2, 2^2, \dots, \lfloor \sqrt{n} \rfloor^2$ 。用 $Q[x]$ 表示凑出 x 最少需要多少个平方数, 依次计算

$$Q[x] = 1 + \min_{1 \leq t \leq \lfloor \sqrt{x} \rfloor} Q[x - t^2].$$

其中初值为 $Q[0] = 0$ 。每个 x 至多尝试 $\sqrt{n}$ 种面额, 因此总时间为 $O(n\sqrt{n})$ 。□

习题 8.12. 若最后的位置 j 不配对, 就只需处理前 $j - 1$ 个位置; 若它与位置 k 配对, 则 k 左边的位置不能与 (k, j) 内部的位置配对, 否则两对会交叉。于是剩下 $[1, k - 1]$ 、 $[k + 1, j - 1]$ 两段独立求解。令 $R[i, j]$ 表示区间 $[i, j]$ 内的最大配对数, 空区间取 0, 则有

$$R[i, j] = \max \left\{ R[i, j - 1], \max_{\substack{i \leq k \leq j-5 \\ s_k, s_j \text{ 可配对}}} (R[i, k - 1] + R[k + 1, j - 1] + 1) \right\}.$$

共有 $O(n^2)$ 个区间, 每个至多尝试 n 个配对位置, 所以时间复杂度为 $O(n^3)$ 。为了恢复配对方案, 可以在计算 $R[i, j]$ 时记录一个 $mate[i, j]$, 表示在最优方案中与 j 配对的 k , 若 j 没有配对, 则记为 NIL。恢复配对方案时, 如果长度不超过 5, 就没有足够的位置容纳一对, 直接返回。对于更长的区间, 若 $mate[i, j] = \text{NIL}$, 说明保留的方案中 j 没有配对, 继续恢复 $[i, j - 1]$ 即可; 否则, 令 $k = mate[i, j]$, 先输出 (k, j) , 再分别恢复 $[i, k - 1]$ 和 $[k + 1, j - 1]$, 因此恢复需要 $O(n)$ 时间。□

习题 8.13. 都不成立：(1) 考虑两条权重均为 1 的边 $s \rightarrow u$ 、 $u \rightarrow v$ ，并按这个顺序扫描。第一条边更新 $d[u] = 1$ ，第二条立即用这个新值更新 $d[v] = 2$ ，则一轮就已经找到了一条含两条边的路线；因此要特别注意一维数组实现与逐个 k 实现的区别；(2) 给出反例：考虑一个包含顶点 1, 2, 3, 4 及单位权边 $1 \rightarrow 3, 3 \rightarrow 4, 4 \rightarrow 2$ 的图。若最外层枚举 i 、中间枚举 j 、最内层枚举 k ，且只执行这三重循环一次，那么计算 $(1, 2)$ 时， $D[1, 4]$ 和 $D[3, 2]$ 都还没有求出， $D[1, 2]$ 仍为无穷。之后虽然求出了这两个中间结果，却不会再更新 $(1, 2)$ ，因此漏掉长度为 3 的路径。□

8.12 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第八讲。
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*, 第 10.3 节：动态规划、最优二叉搜索树与全源最短路。
3. Thomas H. Cormen 等, 算法导论, 第三版, 第 15.1、15.2、15.4、15.5 节：动态规划。
4. Jon Kleinberg、Éva Tardos, *Algorithm Design*, 第 6.1–6.2、6.5–6.9 节：带权区间调度、RNA、序列对齐及最短路。其中第 6.7 节介绍如何用 $O(m + n)$ 空间求出两个字符串的最优对齐方案。
5. Daniel S. Hirschberg, [A Linear Space Algorithm for Computing Maximal Common Subsequences](#), *Communications of the ACM* 18(6), 1975, 341–343。
6. Wolfgang Bein、Mordecai J. Golin、Lawrence L. Larmore、Yan Zhang, [The Knuth–Yao Quadrangle-Inequality Speedup Is a Consequence of Total Monotonicity](#), SODA 2006, 31–40。
7. Richard Bellman, [Dynamic Programming](#), *Science* 153(3731), 1966, 34–37。名称来历另见 Stuart Dreyfus, [Richard Bellman on the Birth of Dynamic Programming](#), *Operations Research* 50(1), 2002, 48–51。
8. G. Malkin, [RFC 2453: RIP Version 2](#), 1998, 第 3.1、3.2、3.4 节：距离向量算法。
9. Ran Duan、Jiayi Mao、Xiao Mao、Xinkai Shu、Longhui Yin, [Breaking the Sorting Barrier for Directed Single-Source Shortest Paths](#), STOC 2025, 36–44。